Computational Physics: Project 1, Part 1

Noah Aney

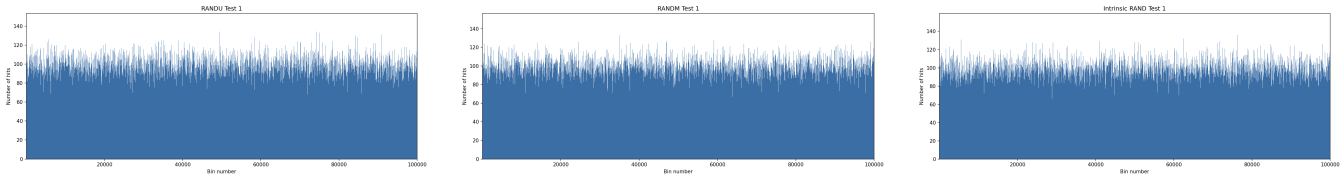
October 24, 2026

The following are the random number generators tested:

- RANDU - From randu.pdf
- RANDM - From example3.f
- RAND - Intrinsic function

Preliminary Testing

We begin by testing these three generators without changing the base code (from rng_test.f), other than implementing the given generators. In this initial test, we have a bin count of 10^5 with a random number sample of 10^7.

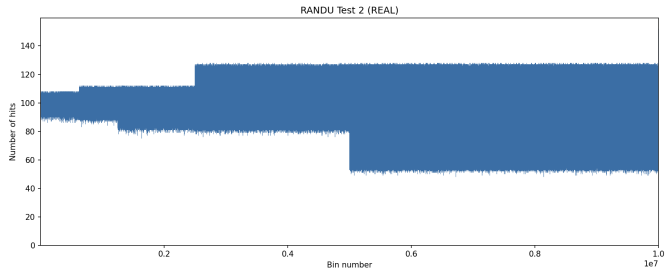


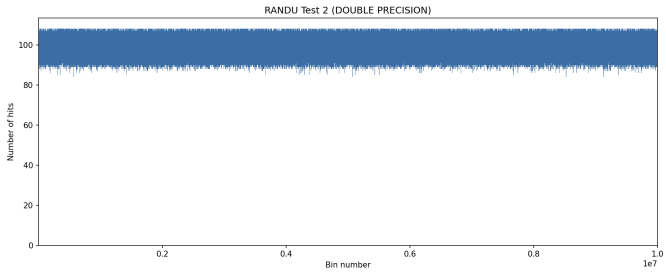
Just by looking at these histograms there is nothing notable about any of these 3. Note that this also follows when looking at the second output file (.out2), as even though there are multiple 4σ bins, we would expect some number of 4σ bins due to the random nature; however, without doing any statistical analysis to determine the expected amount of 4σ bins, this tells us essentially nothing. The essence of the statistically significant deviations that this shows, however, will be visible and made clearer through the use of histograms. So, we turn to using these histograms for the majority of our analysis. As we just saw though, there is something that we must adjust first to get anything worthwhile from these histograms.

Numerical Precision

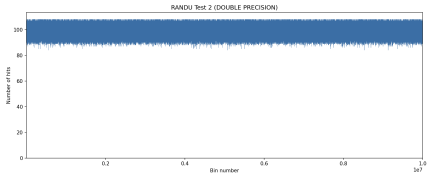
The first thing chosen to change was the numerical precision. What this means is adjustment in BIN numbers as well as changing our random number to DOUBLE PRECISION from REAL. The first plotting done was looking at RANDU with an increased bin count from 10^5 to 10^7, resulting also in an increased random number sample from 10^7 to 10^9 (note too that increasing by too much

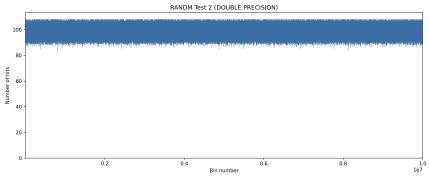
would create a noticeable increase in file size but too little we wouldn't see a big enough effect), comparing the random number, RN1, as REAL vs DOUBLE PRECISION:

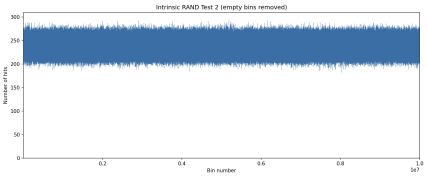




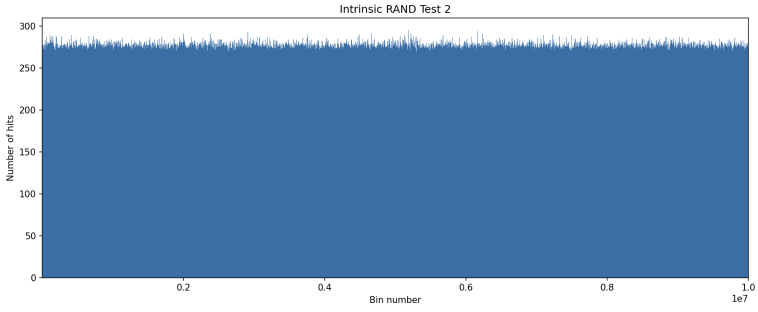
One interesting feature as we increase the bin count is that the standard REAL that we were using introduces these weird steps at given bin numbers (say $0.5 * 10^7$). These steps are a result of rounding error when increasing the precision (result from changing the bin count). This is because the precision starts to encroach on the size of the real number itself, which causes interesting interactions at different rounding intervals. Whatever the exact reasoning, it is clear that there is a need for DOUBLE PRECISION when increasing the bin count. Since we want to increase bin count to get more accurate histograms for behavior analysis, we will be defaulting to DOUBLE PRECISION from here on out. Just to create a new basis then for this increased bin count and use of DOUBLE PRECISION, we can plot the same 3 histograms from Preliminary Testing:







Note that the intrinsic RAND function actually cannot be changed from REAL to DOUBLE PRECISION; however, to account for the increased bin count we removed all bins with hit count of 0 (instead of creating steps due to the REAL rounding, it created bins with a hit count of 0). If interested, this is what the non removed bin RAND histogram looks like:



Now going back and looking at the 3 histograms plotted side-by-side, we can definitely see differences between the 3 generators. As suggested in the project description, we do see a clear similarity between RANDU and RANDM, with their band of number of hits in any given bin being

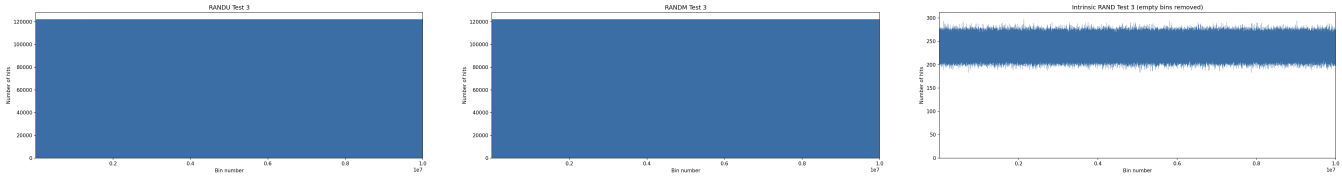
basically the same (roughly 90-110 # of hits). By contrast to RANDU/RANDM, however, the intrinsic RAND function has a much wider band (roughly 200-275 # of hits). The wider band signals to more randomness since the spread (deviation from mean) is larger and thus making each bin count less predictable. This is exactly the statistically significant deviations mentioned earlier. Having a larger breadth of σ values (spread) makes the pattern is less predictable. This gives solid evidence that RAND is a better generator than that of RANDU/RANDM.

Seed Variation

To further validate our findings from the last section that RAND is better than RANDU/RANDM, we try plotting now on a different seed. This whole time we had been using $IX = 777$ (IX being the seed). Now, we will choose a 5 digit even number (for the sake of the project description):

$$IX = 2^{16} = 65536$$

Then, plotting the 3 generators (still using DOUBLE PRECISION), we have:



Here we notice something strange going on with RANDU and RANDM, whereas RAND has the same good pattern as before. What is happening with RANDU/RANDM is due to changing from an odd number to an even number (can be easily checked, as one would expect, that whether or not it's 5 digits doesn't matter). This isn't something explainable without looking at how RANDU/RANDM are made as functions, but it's just a key case to note that they require (to work correctly) an odd seed; however RAND's fundamental pattern won't change under seed variation.
It is clear after all this testing that RAND is our best random number generator (within this sample), where RANDU/RANDM performing similar are not great generators much at all. We should note too that RANDU/RANDM performing so similar is simply a result of how similar their functions are. The only difference between the functions is what constant the random number gets multiplied by at a specific step (from 65539 in RANDU to 6539 in RANDM). Continuing from here, it would be worthwhile to see how our generators fit into our sample, such as R250. Seeing how the pattern changes, especially when compared to RAND, would provide a better image on how to judge the generators as a whole.

Project 1, Part 2: Hunting for Anomalies in Pseudorandom Number Generators

PHYS 25000, Computational Physics

Noah Aney (with Claude)

October 2026

Abstract

We test nine pseudorandom number generators with seven families of methods, going well beyond the one-dimensional histogram of Part 1. They include deep histograms with an exact finite-period theory, triplet correlations and the exact spectral test, the frequency of decimal digits at deep positions, bit-level identities, maximum and minimum spacings, birthday collisions, exact period measurements, and a physics application: the two-dimensional Ising model. Every generator except the two modern ones (MT19937 and gfortran's `RANDOM_NUMBER`) shows at least one statistically decisive anomaly, and nearly every anomaly is explained quantitatively. Highlights: RANDU's triplets lie on 15 planes, which makes a Monte Carlo estimate of the unit-ball volume converge to 3.15432 instead of π ($+60\sigma$); the course's RANDM, whose multiplier differs from RANDU's by one digit, is the better of the two in three dimensions but the worse in two, and it has a detectable lag-one correlation equal to $1/6539$; the intrinsic `RAND()` is the Park-Miller generator truncated to only 2^{22} output values; and the shift-register generator R250, which passes the histogram, digit and maximum-spacing tests, gives a specific heat of the critical Ising model that is wrong by 3.5% (-47σ) under Wolff cluster updates, reproducing the classic result of Ferrenberg, Landau and Wong. The seed question from Part 1 is settled exactly: the period of RANDU or RANDM is 2^{29-k} for a seed with k trailing zero bits, so oddness matters and the number of digits does not.

1 Introduction

In Part 1 we sorted random numbers into histogram bins and looked for statistically significant deviations. Two lessons emerged. First, at a shallow depth (10^5 bins, 10^7 draws) RANDU, RANDM and `RAND()` are indistinguishable. Second, at a deep level (10^7 bins, 10^9 draws) all three misbehave, but in ways that a single histogram cannot fully explain: RANDU and RANDM give bins that are too even, `RAND()` leaves 58% of the bins empty, and storing the numbers in `REAL` adds steps at $x = 1/8, 1/4, 1/2$.

This part asks a broader question: what kinds of nonrandomness hide in these generators, and which test exposes each kind? A pseudorandom generator is a deterministic map on a finite state space, so it always has structure. A good generator hides that structure from every test that matters for the application. We therefore use tests that probe different structures (the marginal distribution, correlations between successive values, the arithmetic of the output, and the finite state space) and finish with a physical simulation in which a subtle correlation produces a wrong physical answer.

As the assignment permits, this part of the project was produced with the assistance of Claude (Anthropic). All code is in the folder `P1_Claude`: the shared generator library `rnglib.f90`, the

rewritten course program `rnd_test_claude.f`, and one subfolder of `tests/` per test family.

2 Generators and implementation

All tests call one Fortran 90 module, `rnglib.f90`, so that every result comes from a single implementation of each generator. Table 1 lists the nine generators. Two of them are deliberate negative controls: von Neumann's middle-square method and the chaotic logistic map.

Table 1: The nine generators. k denotes an integer state; "values" is the number of distinct outputs.

Name	Recurrence	Output	Values	Period
RANDU	$k_{n+1} = 65539\,k_n \bmod 2^{31}$ (IBM)	$k/2^{31}$	2^{29}	2^{29}
RANDM	$k_{n+1} = 6539\,k_n \bmod 2^{31}$ (course `example3.f`)	$k/2^{31}$	2^{29}	2^{29}
RAND	gfortran intrinsic `RAND()` (`example4.f`)	$\lfloor (k-1)/2^9 \rfloor / 2^{22}$	2^{22}	$2^{31}-2$
MINSTD	$k_{n+1} = 16807\,k_n \bmod (2^{31}-1)$ (Park-Miller)	$k/(2^{31}-1)$	$2^{31}-2$	$2^{31}-2$
R250	$w_n = w_{n-103} \oplus w_{n-250}$ on 32-bit words	$w/2^{32}$	2^{32}	$2^{250}-1$
RANNUM	gfortran `RANDOM_NUMBER` (xoshiro256**)	53 bits	2^{53}	$2^{256}-1$
MT19937	Mersenne Twister	53 bits	2^{53}	$2^{19937}-1$
MIDSQ	middle 8 digits of k_n^2	$k/10^8$	$\le 10^8$	short
LOGISTIC	$y_{n+1} = 4y_n(1-y_n)$ in double precision	$\frac{2}{\pi}\arcsin\sqrt{y}$	finite	finite

Validation. Each implementation was checked against published reference values: RANDU from seed 1 gives the states 65539, 393225, 1769499 and from seed 314159 reproduces the triplets of the course handout, with $9x - 6y + z$ exactly integer; MINSTD from seed 1 gives $z_{10000} = 1043618065$, the Park-Miller check value; MT19937 from seed 5489 gives the reference outputs 3499211612 and 581869302, and NumPy's `RandomState(777)` reproduces our stream exactly; R250 satisfies its recurrence for every word. Independent re-implementations in each test family agreed with the library bit for bit.

What `RAND()` really is. Running `RAND()` next to MINSTD from the same seed shows that gfortran's intrinsic is the Park-Miller generator with its output truncated:

$$\texttt{RAND()} = \left\lfloor \frac{k-1}{2^9} \right\rfloor 2^{-22}, \tag{1}$$

with zero mismatches in 2×10^8 draws. It therefore has only $2^{22} = 4{,}194{,}304$ distinct values, and each of them is produced by exactly 512 MINSTD states per period. This single identity explains every `RAND()` anomaly in this report, including the empty bins of Part 1.

Variables in single precision. To emulate a `REAL` variable holding the random number, the library can round every output to IEEE single precision (24-bit mantissa) before returning it. We call this the `REAL*4` mode.

3 Deep histograms and the finite-period law

3.1 Method

The course program was rewritten (`rnd_test_claude.f`) to run any generator, in either precision, with any number of bins B and draws N. It reports compact statistics instead of a per-bin file. The main one is the dispersion index

$$D = \frac{s^2}{\bar{n}}, \qquad \bar{n} = N/B, \tag{2}$$

where s^2 is the variance of the bin counts. For independent uniform draws the counts are Poisson, so $D = 1$ within about $\sqrt{2/B}$. $D < 1$ means the bins are too even, and $D > 1$ too uneven. We also report the χ^2 statistic with its z-score, the number of empty bins, the number of bins beyond 4σ, and the full count-of-counts table (how many bins received exactly k hits). We scanned the depth from $B = 10$ to $B = 10^8$ at 100 expected hits per bin (10 at $B = 10^8$).

3.2 Theory: why the bins are too even

A generator with period P visits each reachable value exactly once per period. Drawing $N < P$ values is therefore sampling *without replacement* from a finite population, and the Poisson variance is reduced by the finite-population factor:

$$D \approx 1 - \frac{N}{P}. \tag{3}$$

More generally, after $N = qP + r$ draws the count in bin b is $n_b = q\,c_b + \mathrm{Hyp}(P, c_b, r)$, where c_b is the number of reachable values in bin b and Hyp is hypergeometric. For `REAL*4` storage, the bin probabilities follow instead from the widths of the single-precision rounding cells. This model has no free parameters.

3.3 Results

Figure 1 is the depth scan. MT19937, RANNUM and R250 stay at $D = 1$ at every depth (largest $|z| = 2.3$ over 24 runs), so the test does not flag good generators. Every other generator fails beyond a depth that the theory predicts. Across all 112 depth-scan points of the non-logistic generators, the measured D agrees with the prediction within 1.95 standard deviations (rms 0.84). The main cases at $B = 10^7$, $N = 10^9$ are:

- **RANDU and RANDM** (period 2^{29}, here $q = 1$): predicted $D = 0.07107$, measured 0.07102 and 0.07105. Their bin counts span only 84 to 108, instead of roughly 60 to 140. RANDU and RANDM visit the same 2^{29} states (odd states congruent to 1 or 3 mod 8) in different orders, so their histograms are statistically identical.

- **MINSTD**: N is about half its period, so $D = 1 - N/P$; predicted 0.5347, measured 0.5348. At $B = 10^8$ only 146 bins are empty, against 4540 ± 67 for Poisson.

- **RAND**: exactly $10^7 - 2^{22} = 5{,}805{,}696$ bins are empty. Each filled bin holds 512 MINSTD states, so its count is Binomial$(512, N/P)$ with mean 238.42 and standard deviation 11.287; we measured 238.42 and 11.284 (Fig. 2).

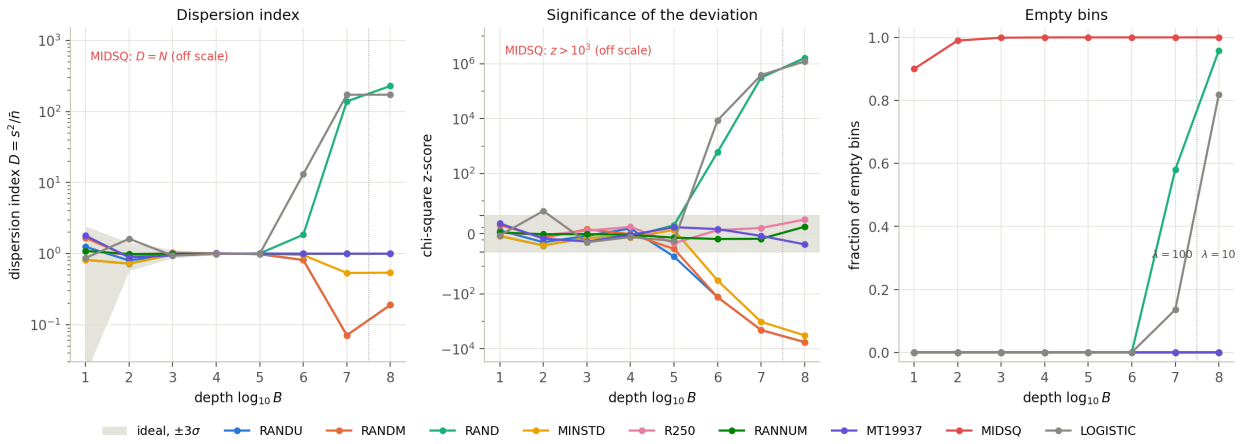


Figure 1: Depth scan with 100 expected hits per bin (10 at $B = 10^8$), seed 777, double precision. Left: dispersion index D. Middle: χ^2 z-score (symmetric log axis). Right: fraction of empty bins. Gray bands are the ideal $\pm 3\sigma$ ranges. MT19937, RANNUM and R250 stay at $D = 1$; RANDU, RANDM (identical curves) and MINSTD drop below 1 as the number of draws approaches their periods, while RAND, LOGISTIC and MIDSQ blow up.

The Part 1 test already detects RANDU. The theory predicts that a generator of period P fails the 100-hits-per-bin test at 3σ once $N \gtrsim 12\,P^{2/3}$, because $z \approx -(N/P)\sqrt{(B-1)/2}$. For the Part 1 configuration ($B = 10^5$, $N = 10^7$) this gives $z = -4.17$ for RANDU and RANDM. Over 200 seeds we measured a mean z of -3.94 (RANDU) and -4.21 (RANDM); seed 777 gives $z = -4.41$ (Fig. 3). The long-period generators scatter around zero. The anomaly is a χ^2 that is too *small*. The course program's criterion (counting bins more than 4σ from the mean) cannot see it, because a too-even histogram has fewer outliers, not more.

Single precision. Storing any generator in `REAL*4` makes the histogram too uneven at $B \geq 10^6$. Between 2^{-g-1} and 2^{-g} the single-precision spacing is constant, so a bin of width $1/B$ contains either $\lfloor w \rfloor$ or $\lceil w \rceil$ representable numbers, where w is the bin width in units of the local spacing. The rounding-cell model predicts $D = 5.45$ at $B = 10^7$ and $D = 31.5$ at $B = 10^8$ for a continuous source, and MT19937, RANNUM and R250 in `REAL*4` give 5.454 and 31.46. Figure 4 shows the resulting steps at every power of two, which are the steps seen in Part 1. This is a property of the storage, not of the generator.

Even seed and negative controls. With the Part 1 seed 65536, RANDU and RANDM reduce to the same 8192 values, and the histogram becomes deterministic: D swings from 0.14 at $B = 100$ to 1.2×10^5 at $B = 10^7$, exactly as predicted. MIDSQ with seed 777 reaches the absorbing state 0 at the second draw ($777^2/10^4 = 60$, $60^2/10^4 = 0$), so it is a constant. The logistic map, supposedly chaotic, turns out to be periodic in double precision: from seed 777 it enters a cycle of 5,638,349 states after 14,324,259 steps (Section 6.3).

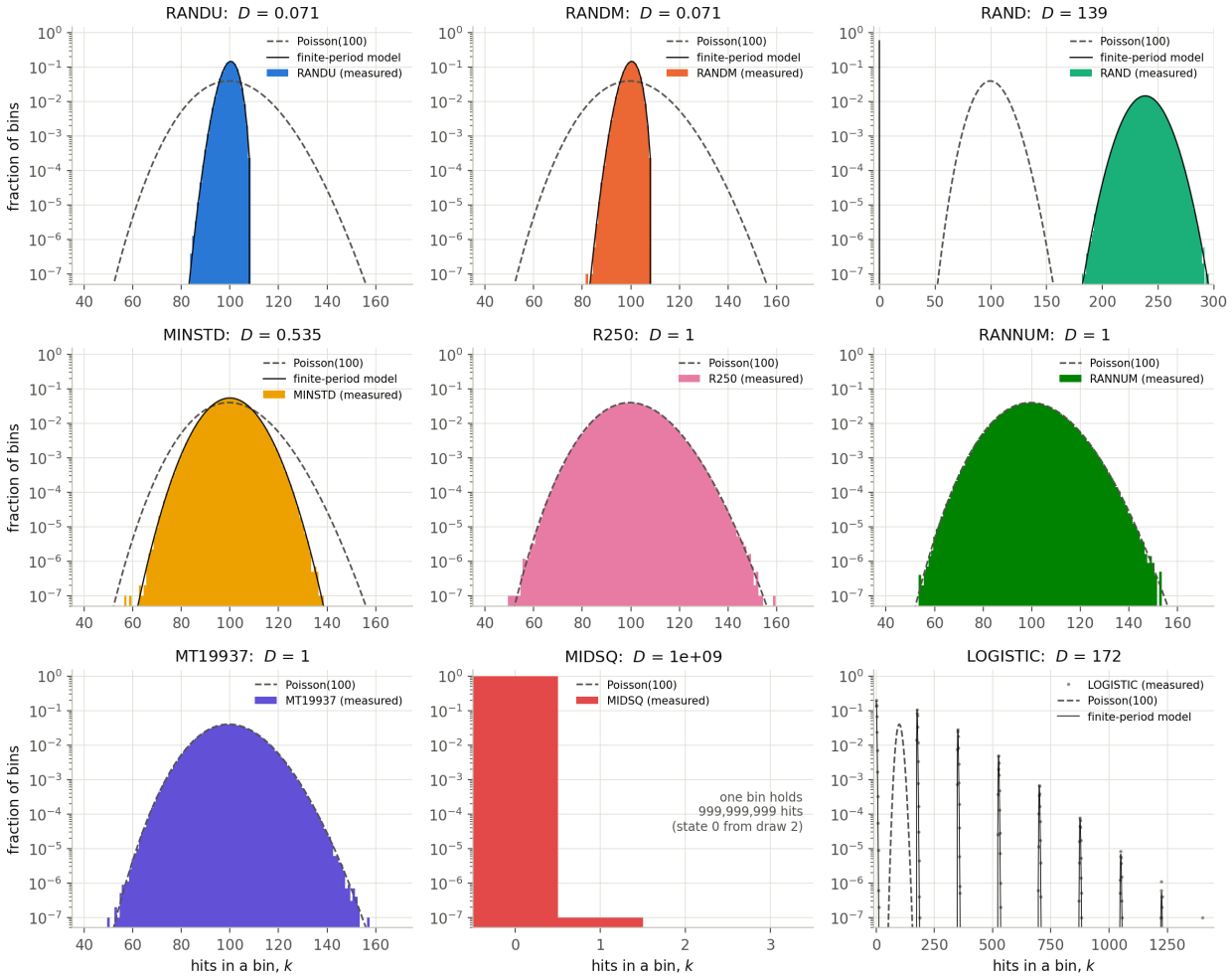


Figure 2: Count-of-counts (fraction of bins receiving exactly k hits) at $B = 10^7$, $N = 10^9$, against the ideal Poisson(100) (dashed) and the finite-period model (solid). This is the fair side-by-side comparison of the generators. RANDU and RANDM are far narrower than Poisson, MINSTD has half the Poisson variance, and RAND splits into a spike of empty bins (58%) and a Binomial(512, 0.466) bump at 238. LOGISTIC forms a comb at multiples of its cycle visits.

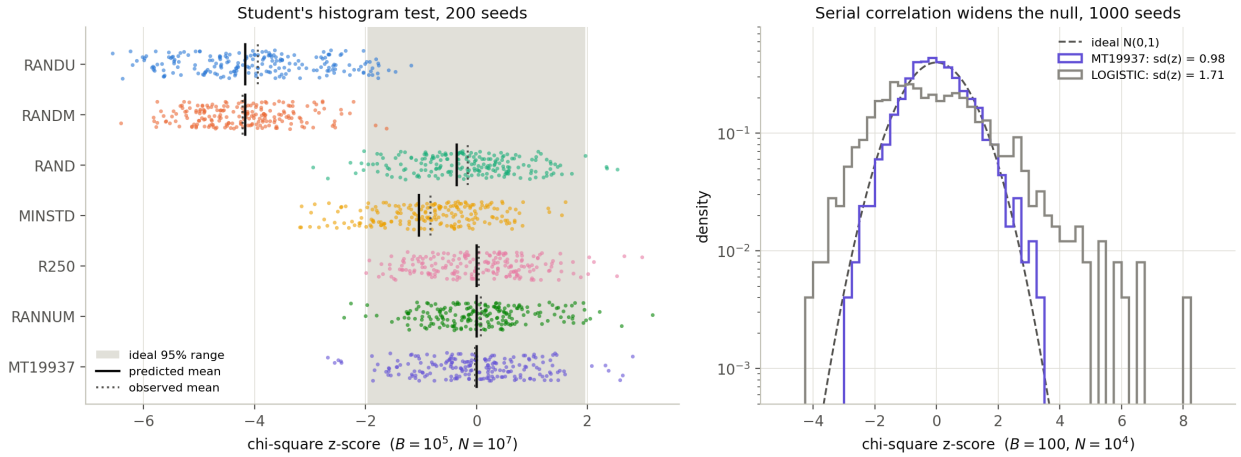


Figure 3: Left: χ^2 z-score of the Part 1 test ($B = 10^5$, $N = 10^7$) for 200 seeds per generator. RANDU and RANDM sit near the predicted $z = -4.17$ (tick) in almost every run, while the long-period generators scatter around 0. Right: at $B = 100$, $N = 10^4$ the logistic map's z distribution is wider than $N(0, 1)$ even though its mean D is 0.99, a signature of serial correlation.

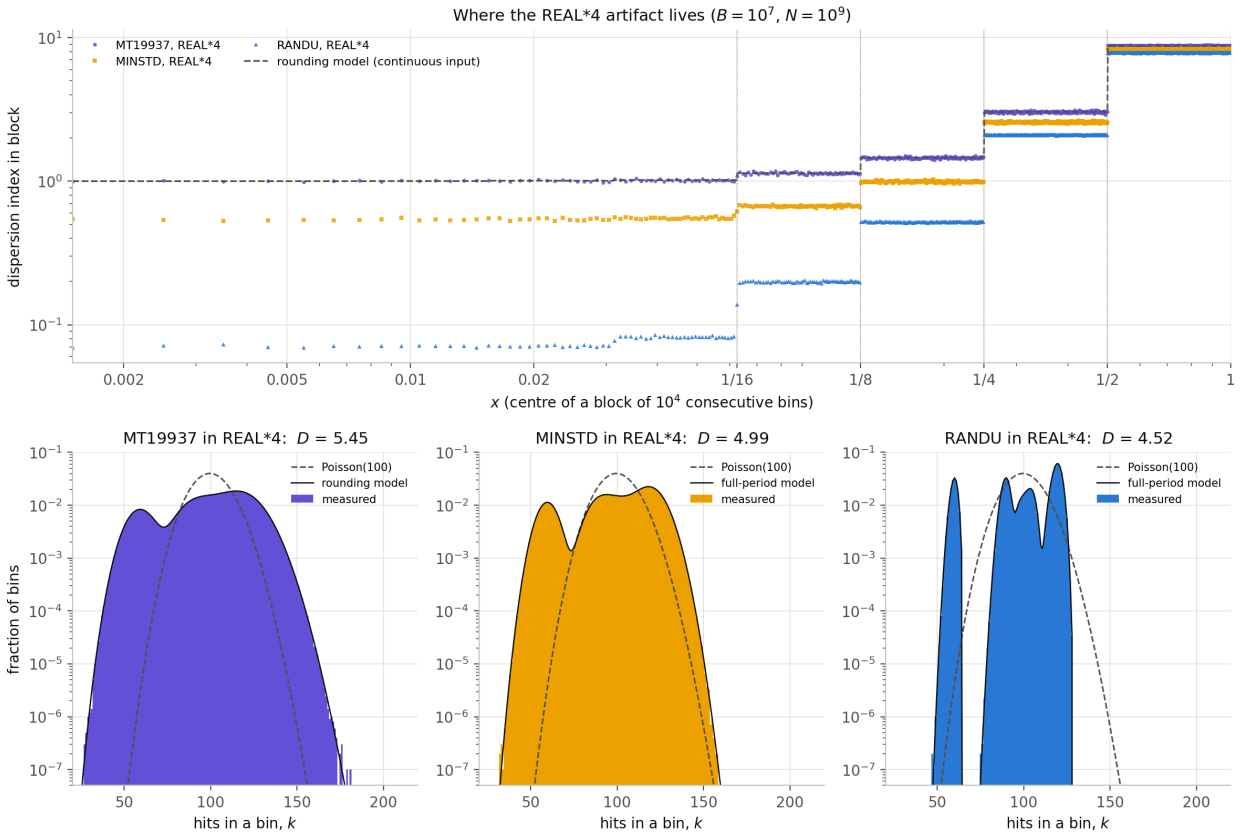


Figure 4: Top: dispersion index within blocks of 10^4 consecutive bins at $B = 10^7$, $N = 10^9$, for numbers stored in `REAL*4`. Steps occur at every power of two, as the rounding model (dashed) predicts; MINSTD and RANDU are shifted down by their period deficits. Bottom: the resulting multimodal count-of-counts; each peak corresponds to bins holding 1, 2, 3 or 4 representable floats.

4 Triplets, hyperplanes and the spectral test

4.1 RANDU's fifteen planes

For RANDU, $a = 65539 = 2^{16} + 3$, so $a^2 \equiv 6a - 9 \pmod{2^{31}}$ and

$$9x_k - 6x_{k+1} + x_{k+2} \in \mathbb{Z}. \tag{4}$$

Every triplet therefore lies on one of 15 parallel planes ($9x - 6y + z = j$, $j = -5, \ldots, 9$), spaced $1/\sqrt{118} = 0.0921$ apart. We verified this in integer arithmetic for 10^7 triplets and over the full period. The planes hold the fractions $(1, 3, 5, 7, 9, 11, 12, 12, 12, 11, 9, 7, 5, 3, 1)/108$ predicted by the geometry ($\chi^2 = 11.6$ on 14 degrees of freedom, $p = 0.64$). Single-precision storage does not remove the planes; it thickens them by only 3×10^{-7} (Fig. 5).

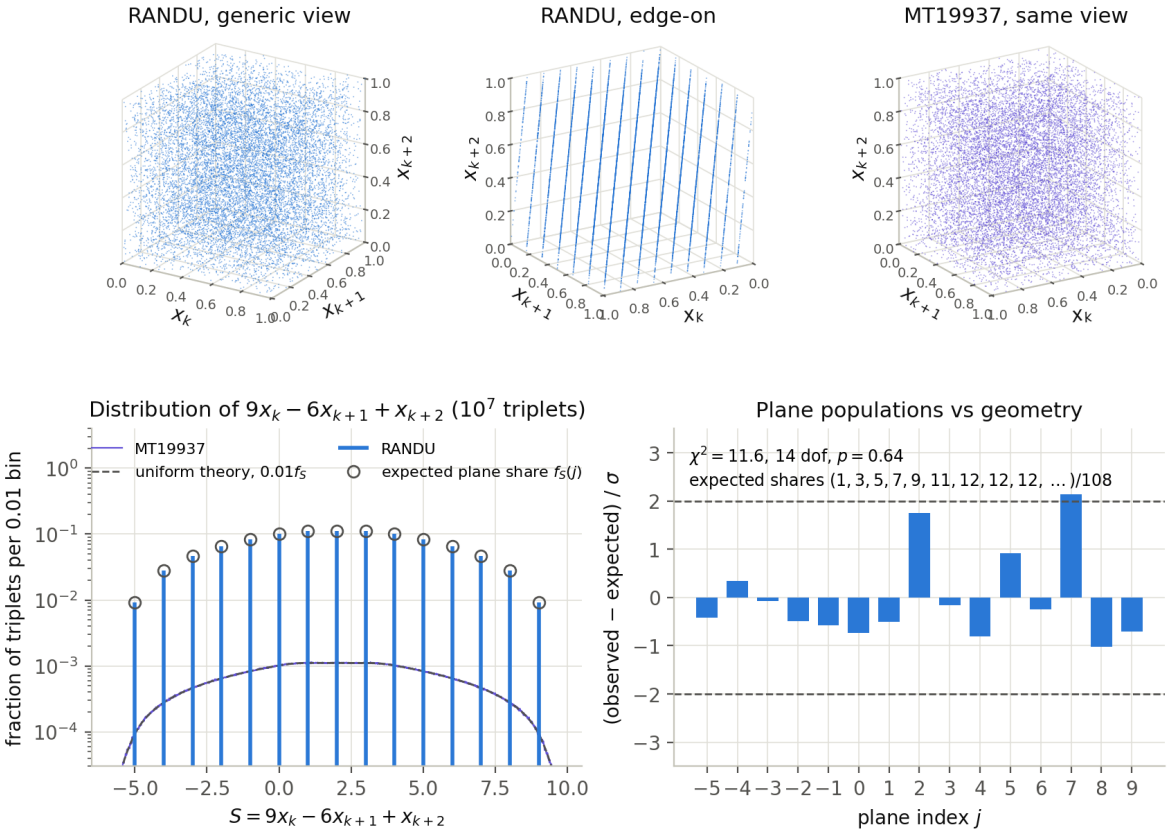


Figure 5: Top: 12000 RANDU triplets seen from a generic angle (no structure visible) and rotated so that the line of sight lies within the planes (15 planes appear), next to MT19937 from the same direction. Bottom left: distribution of $9x_k - 6x_{k+1} + x_{k+2}$ for 10^7 triplets; all RANDU mass falls on the integers -5 to 9, while MT19937 follows the continuous density. Bottom right: plane populations against the geometric shares.

4.2 The exact spectral test

Every multiplicative linear congruential generator (LCG) places its t-tuples on a lattice, which is covered by families of parallel hyperplanes. The spectral test computes ν_t, the length of the shortest vector of the dual lattice; the hyperplanes of that family are $1/\nu_t$ apart. The normalized figure of

merit is

$$S_t = \frac{\nu_t}{\gamma_t^{1/2}\, m^{1/t}}, \tag{5}$$

with Hermite constants γ_t, so that $S_t = 1$ is the best possible lattice. We computed ν_t exactly for $t = 2$ to 8 (LLL reduction followed by exhaustive Fincke-Pohst enumeration in exact integer arithmetic) and confirmed all 43 predicted plane families by enumerating every tuple of the full period.

One subtlety matters for RANDU and RANDM: with $m = 2^{31}$, $a \equiv 3 \pmod 8$ and an odd seed, the reachable states are only those congruent to x_0 or $3x_0$ modulo 8. The correct lattice has modulus 2^{30}, not 2^{31} (which overstates the quality) or 2^{29}.

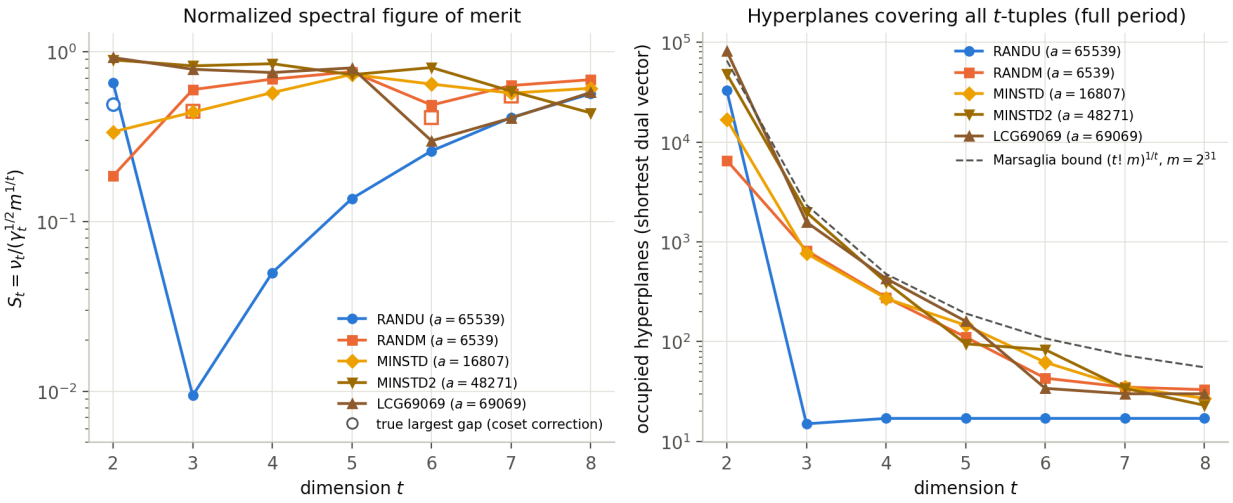


Figure 6: Left: exact normalized spectral figure of merit S_t for $t = 2$ to 8. RANDU collapses at $t = 3$ ($S_3 = 0.0095$), and RANDM is weakest at $t = 2$ ($S_2 = 0.19$); open markers show the true largest gap after the coset correction. Right: number of occupied hyperplanes in the shortest-dual-vector family, compared with Marsaglia's upper bound $(t!\, m)^{1/t}$.

Figure 6 summarizes the result. RANDU has the worst value of all 35 (generator, dimension) pairs: $S_3 = 0.0095$, with 15 planes where Marsaglia's bound allows up to 2344. Its 4D to 6D values are also poor ($S_t \le 0.26$). The reference multipliers are not perfect either: MINSTD (16807) is weakest in 2D ($S_2 = 0.34$), the "improved" Park-Miller multiplier 48271 is weak in 8D, and 69069 is weak in 6D. All LCGs have planes; only the severity differs.

4.3 The "small but significant" difference between RANDM and RANDU

The two multipliers differ by one digit, 6539 versus 65539, and the spectral test shows that they fail in *opposite* dimensions. RANDU is good in 2D ($S_2 = 0.66$) and catastrophic in 3D. RANDM is the worst of the five LCGs in 2D: every pair lies on one of 6539 lines $6539x_k - x_{k+1} = j$, and whenever $x_k < 1/6539$ the next value is exactly $x_{k+1} = 6539\, x_k$ (Fig. 7). From 3D upward, however, RANDM is far better than RANDU: its 3D planes are the 818 planes of $25x_k + 115x_{k+1} + 678x_{k+2} \in \mathbb{Z} + \frac{1}{2}$, giving $S_3 = 0.60$.

The weak 2D lattice of RANDM has a direct statistical consequence. For a multiplicative LCG, the lag-one serial correlation is approximately $1/a$. Computing it exactly over RANDM's full period gives $r_1 = 1.5293 \times 10^{-4} \approx 1/6539$, which over $N = 2^{29}$ values is a 3.5σ effect (Fig. 8). For RANDU,

$r_1 = 1.36 \times 10^{-5}$ is ten times smaller and undetectable (0.3σ). So RANDM has a measurable pair correlation that RANDU lacks, while RANDU has a triplet structure that RANDM lacks.

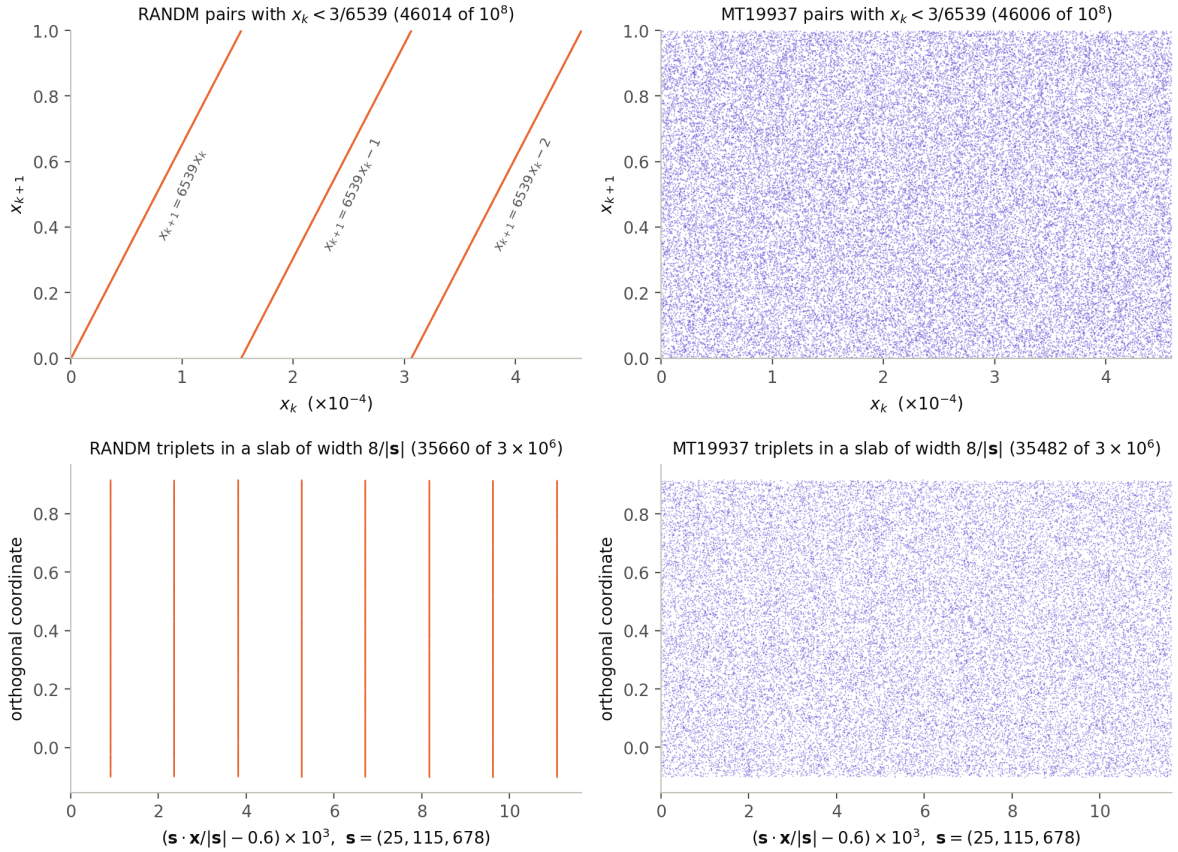


Figure 7: Top: RANDM pairs with $x_k < 3/6539$ lie exactly on the lines $x_{k+1} = 6539x_k - j$ (RANDM's 2D weakness), while MT19937 fills the same strip uniformly. Bottom: RANDM triplets in a slab eight plane spacings wide along $\mathbf{s} = (25, 115, 678)$ fall on 8 planes, so its 3D lattice is about 75 times finer than RANDU's.

4.4 Monte Carlo integrals that the lattice breaks

Each LCG has an elementary integrand that it gets completely wrong. If $\mathbf{s}$ is a shortest dual vector, then

$$I = \int_{[0,1)^t} \cos\bigl(2\pi\, \mathbf{s} \cdot \mathbf{x}\bigr)\, d\mathbf{x} = 0, \tag{6}$$

but for the owning generator $\mathbf{s} \cdot \mathbf{x}$ is always an integer (for RANDU, $\mathbf{s} = (9, -6, 1)$), so the Monte Carlo estimate is exactly 1 with zero sample variance at every N. At $N = 10^7$ this is a 4472σ error, and the naive error bar is zero, so the wrong answer is reported with infinite confidence (Fig. 9). Each killer integrand fails only for its own generator; MT19937 and RANNUM pass all of them.

Less contrived 3D calculations fail too (Fig. 10):

- **Volume of the unit ball.** Hit-or-miss Monte Carlo with RANDU triplets converges to $\hat{\pi} = 3.15426 \pm 0.00021$ at 2×10^8 triplets, 60σ above π. With RANDU, 3D Monte Carlo is really a quadrature on 15 planes, whose exact value 3.1543170 we reproduce analytically. RANDM, MINSTD and MT19937 give π within 1σ.

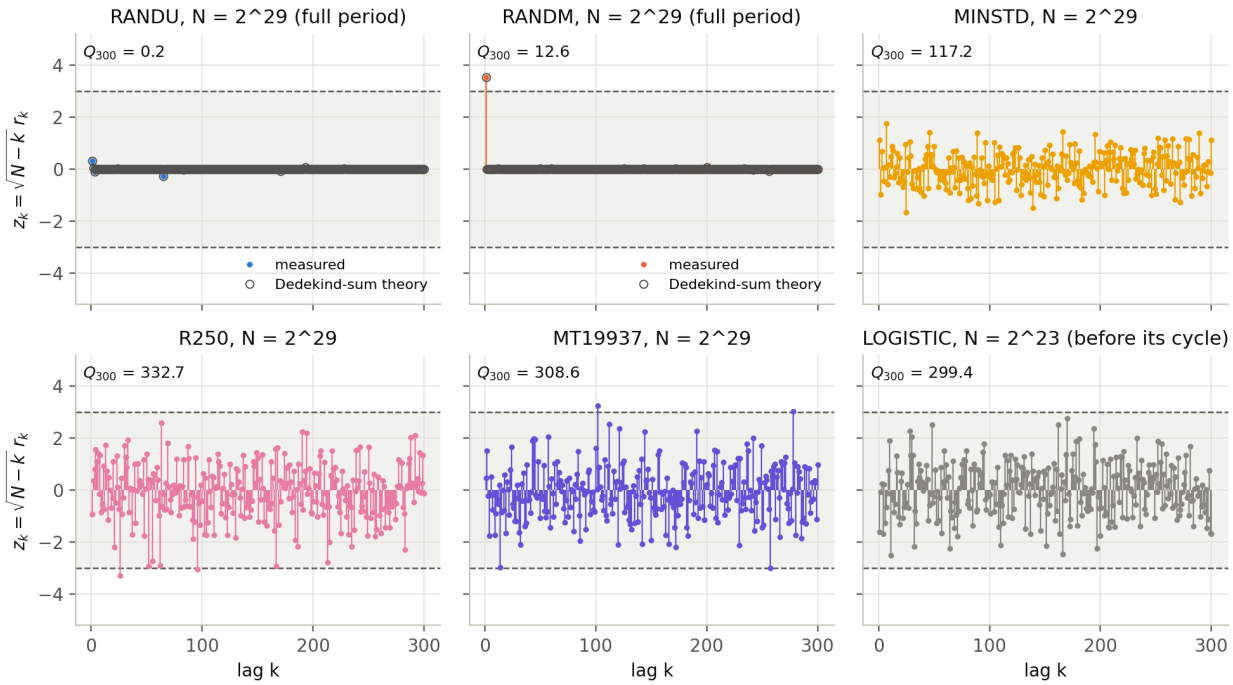


Figure 8: Lag-k serial correlation in units of its standard error, $z_k = \sqrt{N-k}\, r_k$, for $k = 1$ to 300 (dashed lines: ± 3). For RANDU and RANDM the sums run over the full period, and the measured values (filled) coincide with the exact Dedekind-sum theory (open circles). RANDM has $z_1 = 3.5$, i.e. $r_1 \approx 1/6539$; RANDU's corresponding value is ten times smaller. The full-period sums are also too close to zero at all other lags ($Q_{300} = \sum z_k^2$ of 0.2 and 12.6 instead of about 300), another face of period exhaustion.

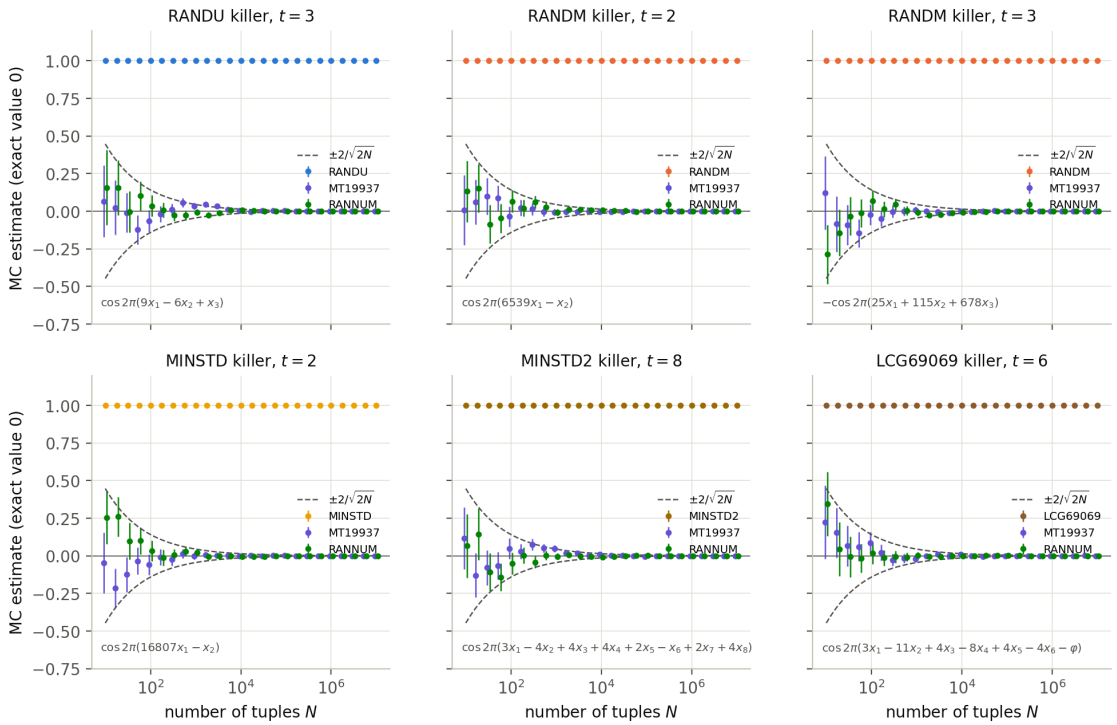


Figure 9: Monte Carlo estimates of $\int \cos 2\pi(\mathbf{s} \cdot \mathbf{x} - \varphi)\, d\mathbf{x} = 0$ built from each LCG's shortest dual vector, against the number of tuples N. The owning LCG returns exactly 1 with zero error bar at every N; MT19937 and RANNUM converge to 0 inside the $\pm 2/\sqrt{2N}$ band.

- **Isotropic directions.** Directions generated by rejection from the cube are grossly anisotropic: 13.75% of all directions lie exactly on the great circle perpendicular to $(9, -6, 1)$, and the band $0 < |\cos\theta| < 2/\sqrt{118} = 0.184$, which covers 18.4% of the sphere, receives no directions at all.
- **An ideal gas that diffracts.** Particles placed at RANDU triplets produce a Bragg peak in the structure factor, $S(9, -6, 1) = N$ exactly, as if the gas were a crystal.

5 Decimal digits and bits

5.1 Digits at deep decimal positions

For each generator we took the decimal digit $d_p = \lfloor x\, 10^p \rfloor \bmod 10$ at positions $p = 1$ to 60 for $N = 10^8$ outputs and tested its frequency with a χ^2 test (9 degrees of freedom). A first finding concerns the test itself: computing $\lfloor x\, 10^p \rfloor$ in floating point fails for *every* generator, MT19937 included, at $p = 16$ in double precision and $p = 8$ in `REAL*4`, because $x\, 10^p$ then exceeds 2^{53} (or 2^{24}) and rounds to even integers. All results below therefore use exact integer arithmetic on the exact value of each output.

With exact digits, a source with B-bit resolution first fails not at the naive depth $B \log_{10} 2$ but at

$$p^* \approx B - \tfrac{1}{2} \log_2 N + 1.3, \tag{7}$$

which is about $B - 11$ for $N = 10^8$. The measured breaking depths agree (Fig. 11): RAND ($B = 22$) fails at $p = 10$, R250 ($B = 32$) at 21, MT19937 and RANNUM (53 bits) at 42, and any generator

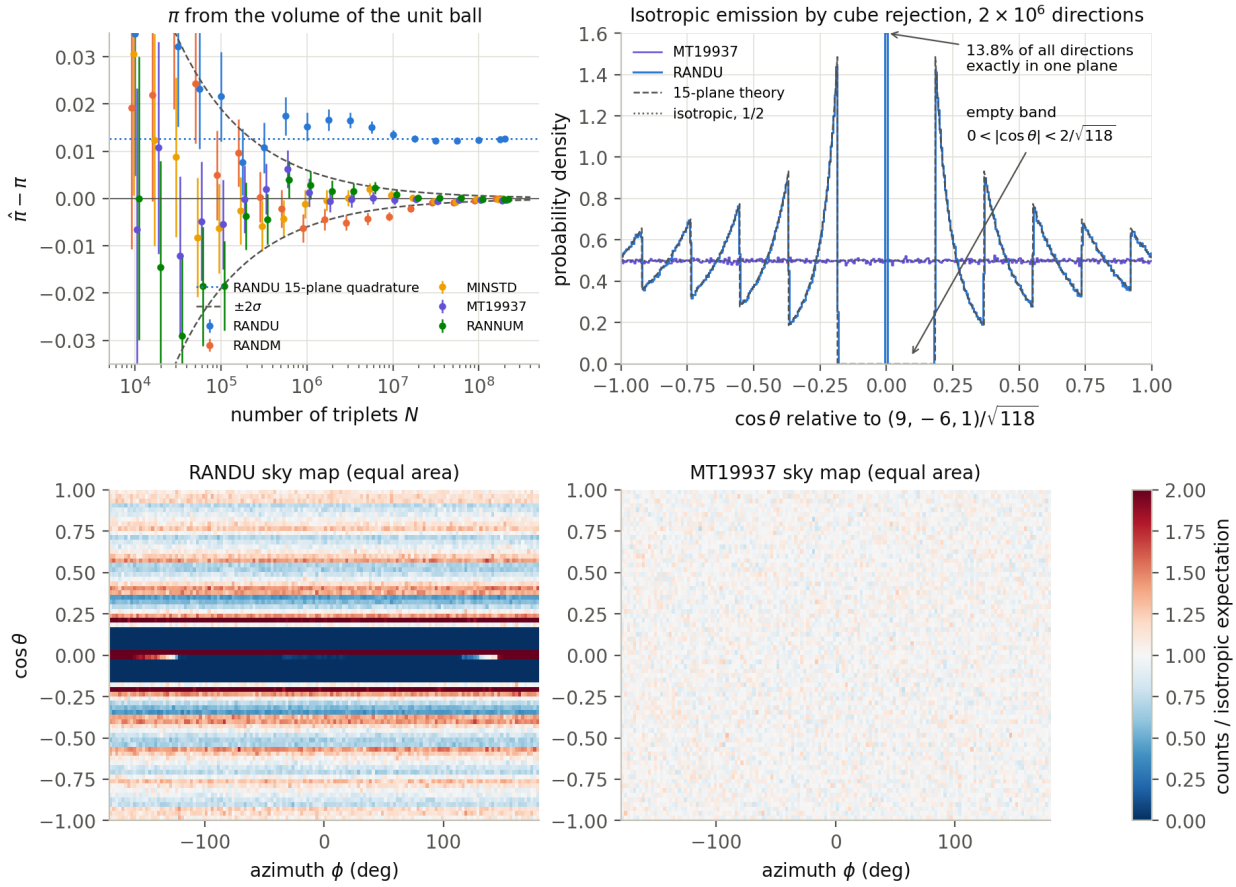


Figure 10: Top left: hit-or-miss estimate of π from the volume of the unit ball; RANDU converges to the 15-plane quadrature value 3.15432 (dotted), not to π. Top right and bottom: directions from cube rejection with RANDU follow the 15-plane theory, with 13.8% of all directions on one great circle and an empty belt $0 < |\cos\theta| < 0.184$; the MT19937 sky map is uniform.

stored in `REAL*4` at 13. The exact rational digits of MINSTD never fail.

RANDU and RANDM behave differently. Their digits are exactly periodic, with period 2^{30-p} in the digit d_p, so within a period every digit appears almost exactly equally often. The digit test is therefore *too uniform* at $p = 5$ to 14 (χ^2 as small as 0.017 where 9 is expected; left-tail p-values down to 10^{-11}), and deterministically nonuniform from $p = 17$. This is the decimal analogue of the too-even histogram bins.

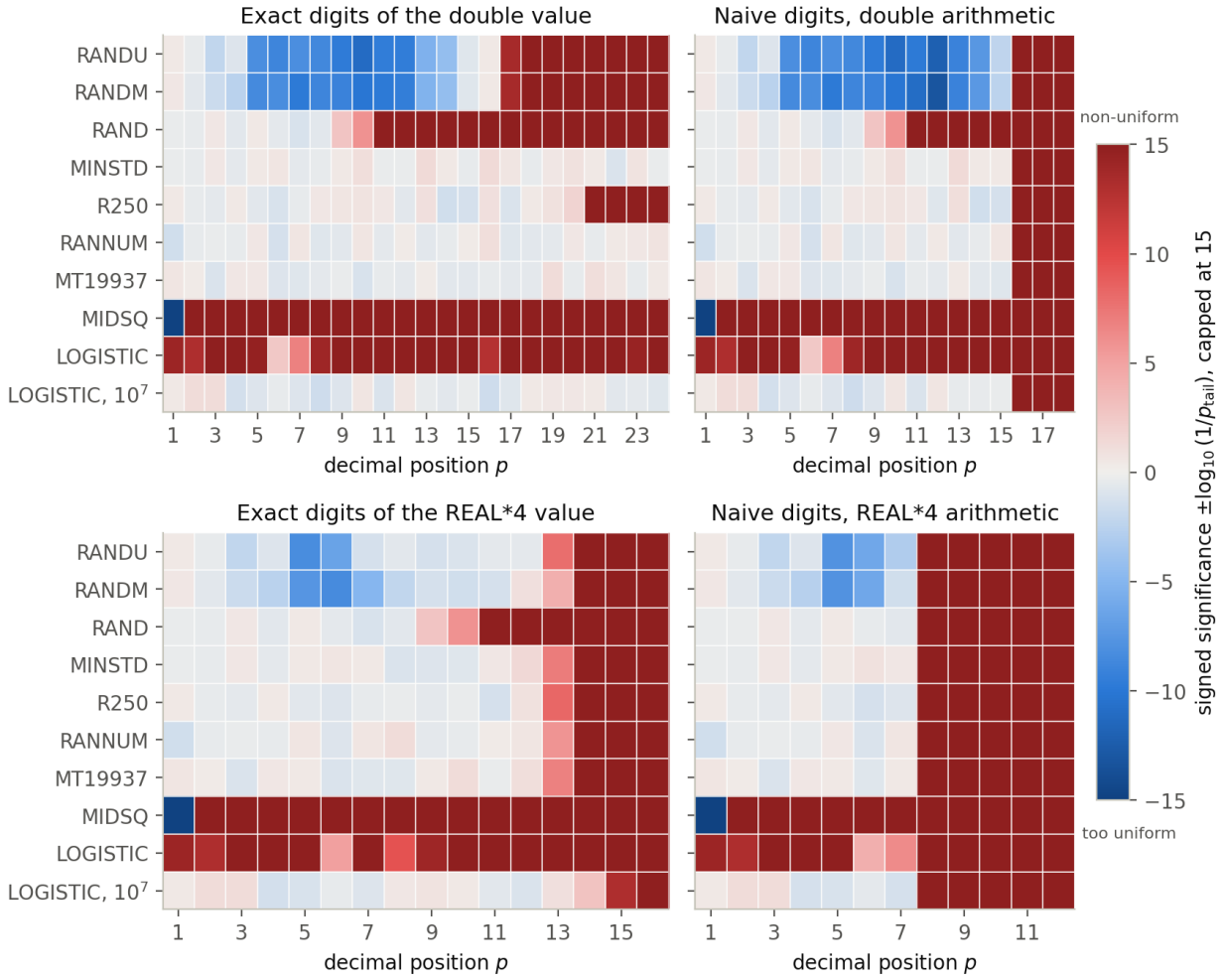


Figure 11: Signed significance of the digit-frequency χ^2 test ($N = 10^8$) at each decimal position p. Red is nonuniform, blue is too uniform, near-white is consistent with uniform. Exact digits fail at a depth set by each generator's resolution: RAND at 10, R250 at 21, RANDU and RANDM at 17 after a too-uniform band at $p = 5$ to 14. Naive floating-point extraction fails for every generator at $p = 16$ in double and $p = 8$ in `REAL*4`. LOGISTIC fails everywhere at 10^8 draws because of its cycle.

5.2 Bit-level structure

Writing $x = \sum_j b_j 2^{-j}$, the low-order bits of a power-of-two-modulus LCG are far from random. For RANDU and RANDM, bit j of $k/2^{31}$ has period exactly 2^{30-j} for $j \le 28$; bit 29 is stuck at 0, bit 30 alternates, and bit 31 is stuck at 1. Because $65539 \equiv 3 \pmod{2^{16}}$, the low 16 bits of RANDU's state evolve by multiplication by 3, which gives bits 16 to 27 a lag-one autocorrelation

of almost exactly $+1/3$ (measured 0.3333 and 0.3335 for bits 16 and 17, matching the full-period theory to six decimals). The Berlekamp-Massey algorithm, which finds the shortest linear feedback shift register that generates a bit stream, gives a linear complexity of exactly $2^{29-j}+1$ for RANDU bit j (Fig. 12).

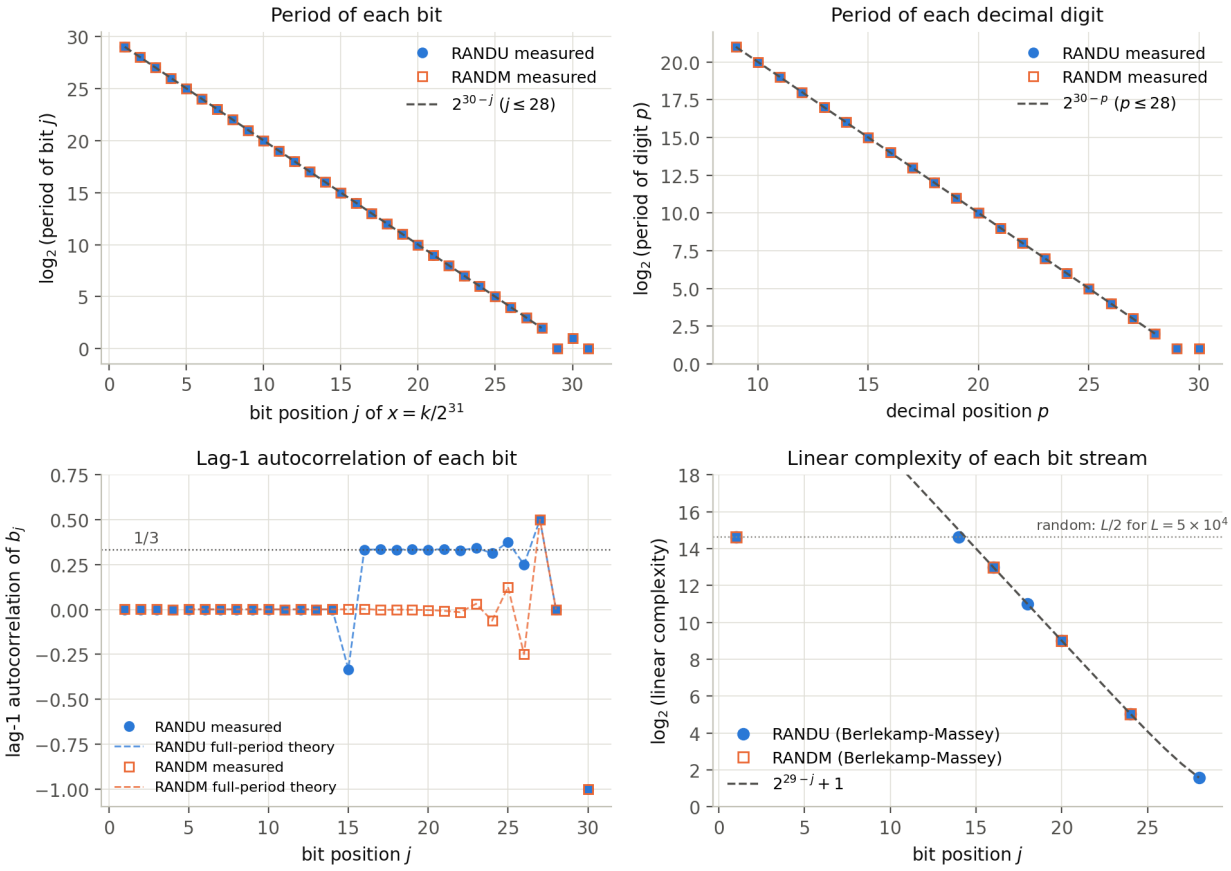


Figure 12: Exact low-order structure of RANDU and RANDM. (a) Measured period of bit j, equal to 2^{30-j}. (b) Period of the decimal digit d_p, equal to 2^{30-p}. (c) Lag-one bit autocorrelation, measured (markers) and computed over the full orbit (dashed); the 1/3 plateau comes from $65539 \equiv 3 \pmod{2^{16}}$. (d) Berlekamp-Massey linear complexity, equal to $2^{29-j}+1$.

Three further bit identities are exact (Fig. 13):

- **R250** obeys $b_j(n) \oplus b_j(n-103) \oplus b_j(n-250) = 0$ in all 32 bits with zero violations in 5×10^7 tests per bit, against 50% for MT19937. A blind scan of all 179,700 lag pairs up to 600 rediscovers exactly $(103, 250)$ and its square $(206, 500)$, and nothing else. Its linear complexity is exactly 250 (and exactly 19937 for MT19937, which is also linear over GF(2)).
- **LOGISTIC**'s output is, bit for bit, the previous output shifted by one place: $b_j(n) = b_{j+1}(n-1) \oplus b_1(n-1)$, exactly for $j \le 26$. This follows from its conjugacy to the tent map. Each 53-bit output therefore carries only about 5 new bits, yet the standard per-bit lag-one test does not see it.
- **MINSTD** stored as a double copies bits 1 to 22 into bits 32 to 53, because $k/(2^{31}-1)$ has a binary expansion of period 31. The copy is broken only by the rounding carry at bit 53.

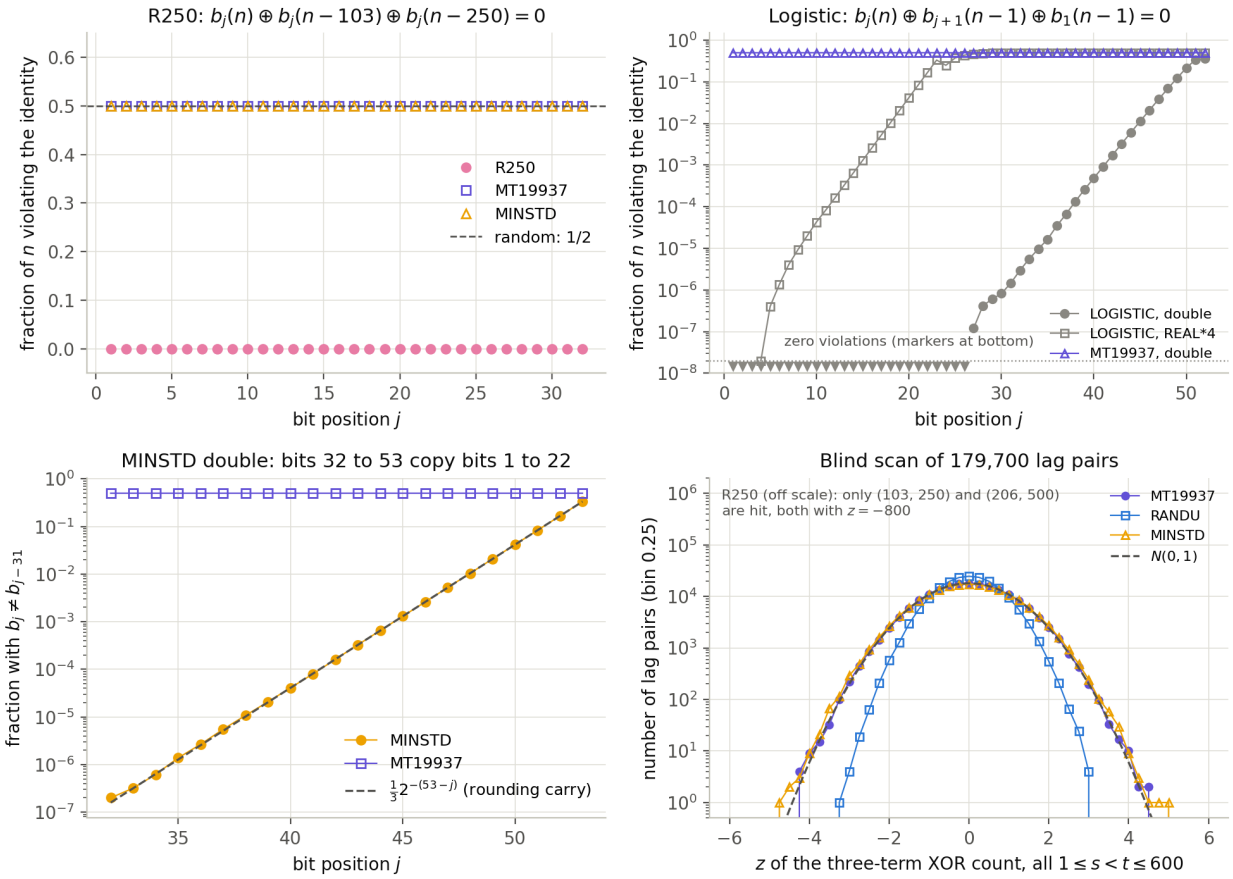


Figure 13: Hidden bit identities. (a) R250 obeys its recurrence in every bit with zero violations, while the controls give 1/2. (b) The logistic output is a one-bit shift of the previous output, exactly in bits 1 to 26. (c) MINSTD doubles copy bits 1 to 22 into bits 32 to 53, broken only by the rounding carry. (d) A blind scan of 179,700 lag pairs flags only the two R250 lags.

6 Spacings, collisions and periods

6.1 Maximum spacing

The assignment suggests the maximum-spacing method of J. Heinrich. We could not retrieve the original CDF note (it is cited in the Particle Data Group review of Monte Carlo techniques), so we implemented the test from first principles. Sort n uniform points and let M_n be the largest gap. Under the null hypothesis

$$P(M_n \le x) = \sum_{k\ge 0} (-1)^k \binom{n+1}{k} (1-kx)_+^n, \qquad T = (n+1)M_n - \ln(n+1) \;\longrightarrow\; \text{Gumbel}. \tag{8}$$

We compared the distribution of T with the exact law for up to 10^4 replicates per sample size, for N from 10^3 to 2^{31} draws.

MT19937 and RANNUM pass every block up to $N = 10^9$ (smallest $p = 0.012$ among 24 blocks, which is not significant after correcting for multiple tests). For the LCGs, the maximum spacing detects period exhaustion rather than the lattice. RANDU drifts low already at $N = 10^8$ (mean $T = -0.81$ against the null 0.58 and our prediction -0.85) and collapses to the deterministic value $T = -13.06$ at $N = 2^{29}$, where every reachable point appears once. It detects output grids only when N is comparable to the number of grid values: `RAND()` fails at $N = 10^7$ (KS $p = 2 \times 10^{-9}$) and `REAL*4` storage at 10^8 (Fig. 14).

6.2 Minimum spacing and birthday collisions

The opposite extreme, the minimum spacing and the number of repeated values, is four to five decades of N more sensitive to the output resolution. A source with G equally likely values produces about $N^2/2G$ repeats among N draws, so counting repeats measures an effective resolution G_{eff} (Fig. 15). The measurements recover the true resolutions to 0.02% to 0.5%: `RAND()` gives $G_{\text{eff}} = 4.2023 \times 10^6 = 2^{31}/511$ (each value covers 512 MINSTD states that never repeat within a period), R250 gives 2^{32}, and `REAL*4` storage gives 0.75×2^{25}.

The LCGs fail this test in the opposite direction. Because their full state is the output, they produce *no* repeated value within a period: RANDU gave 0 repeats in 300 samples of 10^6 draws, where 2.8×10^5 are expected for independent draws from 2^{29} values. Real random numbers have birthdays; RANDU never does. The logistic map fails both spacing tests already at $N = 10^3$: its tent-map dynamics turns every close pair into a chain of pairs whose distance doubles at each step, which makes $E[n(n+1)m_n] = 2$ exactly instead of 1. In a three-dimensional birthday-spacings test (Marsaglia's version on triplets), RANDU fails with only 256 points.

6.3 Exact periods and the seed question

We measured periods exactly with Brent's cycle-detection algorithm on the full internal state. For RANDU and RANDM all odd seeds, from 3 to 314159 (one to six digits), give the full period $2^{29} = 536{,}870{,}912$. A seed of the form $2^k \times$ (odd) gives

$$\lambda = 2^{29-k} \tag{9}$$

for every $k = 1$ to 24 without exception, and seed 0 is a fixed point (Fig. 16). The Part 1 seed $65536 = 2^{16}$ therefore has period $2^{13} = 8192$. The answer to the assignment's question is that a five-digit seed is superstition but an odd seed is essential: each factor of 2 in the seed halves the

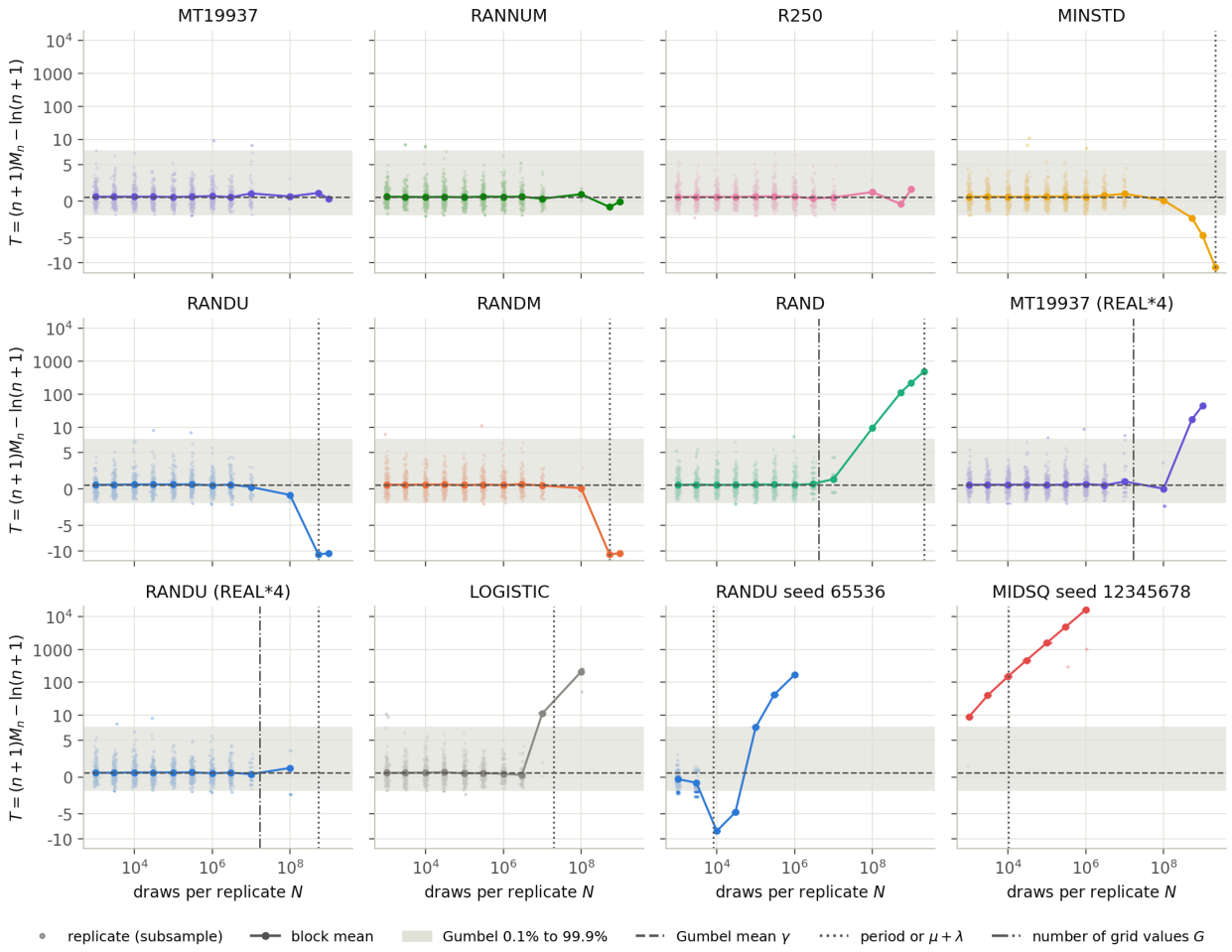


Figure 14: Maximum-spacing statistic T against the number of draws per replicate (points: replicates; line: block mean; band: Gumbel 0.1% to 99.9%). Good generators stay in the band up to $N = 10^9$. LCGs fall below it as N approaches their period (dotted), and grid-limited outputs (RAND, `REAL*4`) rise above it once N exceeds the number of grid values (dash-dot).

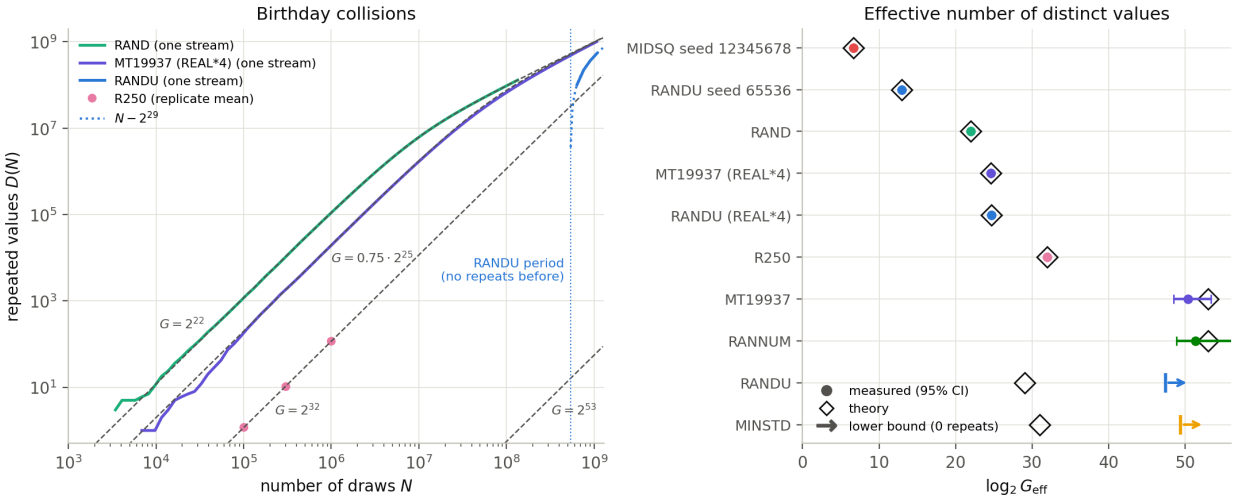


Figure 15: Left: number of repeated values $D(N)$ against the birthday prediction for G equally likely values (dashed). RANDU produces no repeat at all before its period 2^{29} and exactly $N - 2^{29}$ afterwards. Right: effective number of distinct values G_{eff} with 95% intervals; the permutation generators give only lower bounds, which exceed their true number of states by at least 2^{18}.

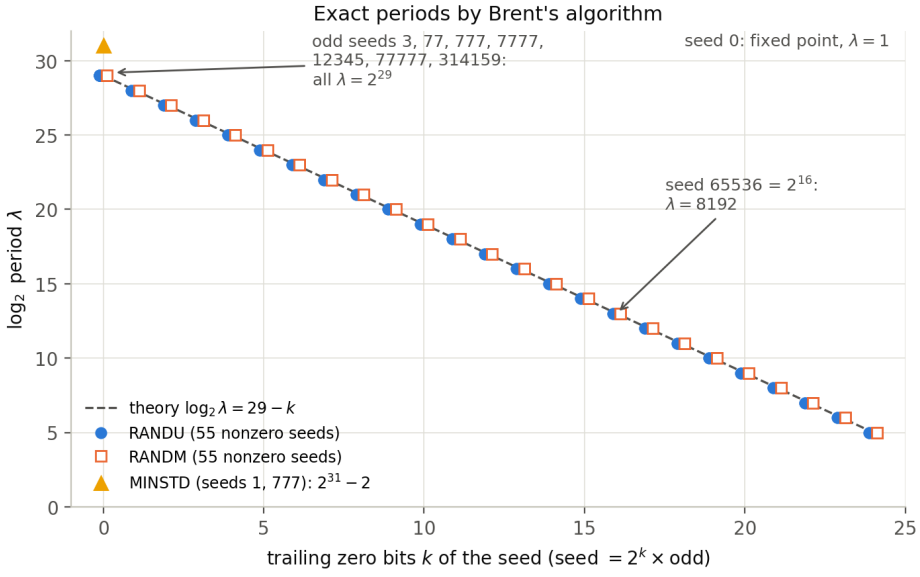


Figure 16: Exact periods from Brent's algorithm for 56 seeds each of RANDU and RANDM: $\lambda = 2^{29-k}$ for seed $2^k \times$odd, independent of the number of digits. The Part 1 seed 65536 has $k = 16$ and period 8192.

period, because the multiplication preserves the trailing zero bits. MINSTD, with prime modulus, has the full period $2^{31} - 2$ for every seed.

The two negative controls have strikingly small state spaces (Fig. 17). A complete enumeration of all 10^8 seeds of the 8-digit middle-square method finds only 17 terminal cycles; half of all seeds end at 0, and no seed produces more than 16,903 distinct numbers. The double-precision logistic map is eventually periodic, as every map on a finite set must be: from 1001 random starts we found six attractors, the most common being a cycle of 5,638,349 states (66.6% of starts), and 17.9% of starts collapse to the fixed point 0, after which the generator outputs 0 forever.

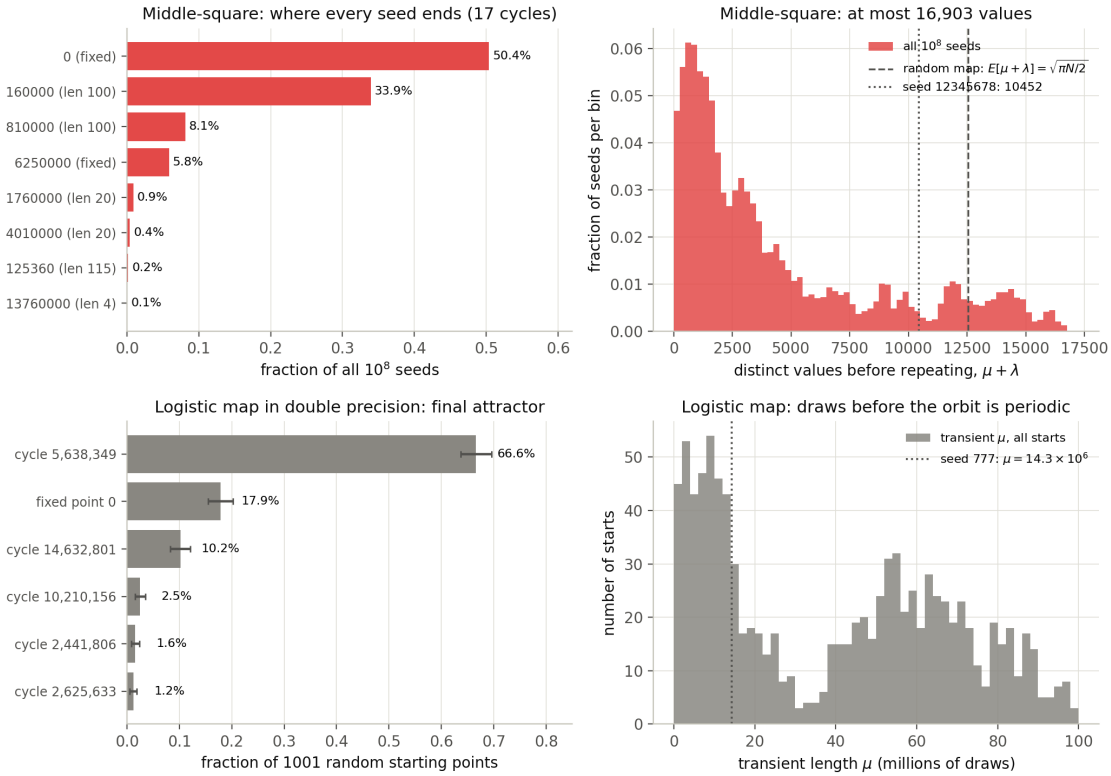


Figure 17: Top: complete enumeration of all 10^8 middle-square seeds: only 17 terminal cycles, half of all seeds end at 0, and no seed yields more than 16,903 distinct numbers. Bottom: 1001 random starts of the double-precision logistic map end in one of six attractors, and 17.9% collapse to 0.

7 A physical test: the two-dimensional Ising model

Distributional tests can be passed by a generator that still ruins a simulation. The classic demonstration is due to Ferrenberg, Landau and Wong (1992): the R250 shift register, which passed the standard test batteries of its day, gives wrong answers for the 2D Ising model when used with the Wolff cluster algorithm. We reproduced their experiment.

Setup. The model is an $L \times L$ periodic square lattice at the critical coupling $K_c = \frac{1}{2}\ln(1+\sqrt{2})$, with energy per spin $e = -\langle \sum_{\langle ij \rangle} s_i s_j \rangle / L^2$ and specific heat C. Exact finite-lattice values follow from the solution of Kaufman (1949) and Ferdinand and Fisher (1969). We implemented it in high precision and validated it against brute-force enumeration of small lattices (agreement to 60 digits).

For $L = 16$:

$$e = 1.4530648528, \qquad C = 1.4987049594. \tag{10}$$

The Wolff algorithm grows a cluster from a random site, adding each aligned neighbor with probability $p_{\text{add}} = 1 - e^{-2K_c} = 2 - \sqrt{2} \approx 0.586$, one random number per bond test. Error bars come from batch means, with the batch length checked against the measured autocorrelation time.

Results. MT19937 and RANNUM agree with the exact values under both algorithms. With 10^7 Wolff clusters, R250 gives

$$e = 1.455032(119), \qquad C = 1.44654(112), \tag{11}$$

that is $\Delta e = +1.97 \times 10^{-3}$ $(+16.5\sigma)$ and $\Delta C = -0.052$ $(-46.7\sigma$, a 3.5% error). The bias does not depend on the seed or the cluster growth order. The same R250 stream drives Metropolis single-spin updates without any detectable bias, $\Delta e = (0.3 \pm 1.9) \times 10^{-4}$ (Fig. 18).

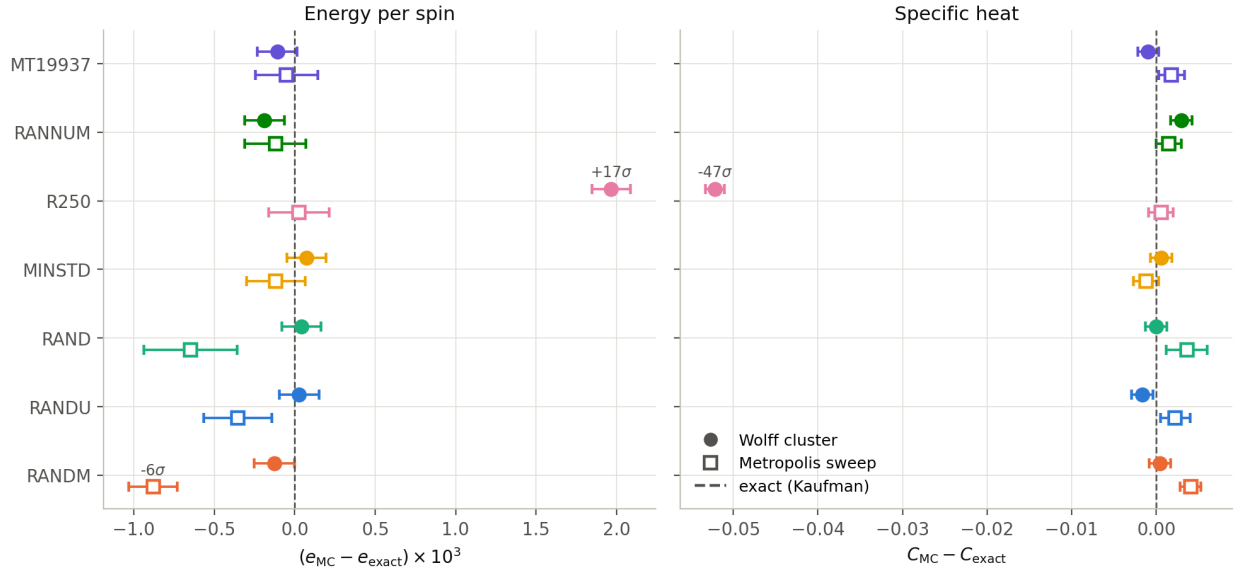


Figure 18: Measured minus exact energy per spin (left, units of 10^{-3}) and specific heat (right) for the 16×16 Ising model at K_c, per generator, for Wolff cluster updates (filled circles) and Metropolis sweeps (open squares), with $\pm 1\sigma$ error bars. Only R250 under Wolff is far off ($+17\sigma$ in e, -47σ in C), while the same R250 is unbiased under Metropolis. The RANDM Metropolis point is discussed in the text.

Mechanism. Because $u_n = u_{n-103} \oplus u_{n-250}$ bitwise, the leading bits multiply. A rejected bond test has $u \geq 0.586 > 1/2$, i.e. a leading bit of 1. If the tests 103 and 250 draws earlier were both rejected, the current draw has a leading bit of 0, so $u < 1/2 < p_{\text{add}}$ and the bond is accepted with probability exactly 1, instead of 0.586 (Fig. 19). This is a large third-order correlation among accept/reject outcomes ($\kappa_3 = 0.071$, about 4200 standard errors), with no pair correlation at all. Cluster growth uses roughly 240 random numbers per cluster on the 16×16 lattice, so the draws 103 and 250 steps back usually belong to the same cluster; in a Metropolis typewriter sweep they belong to fixed, distant sites, and the correlation does no harm.

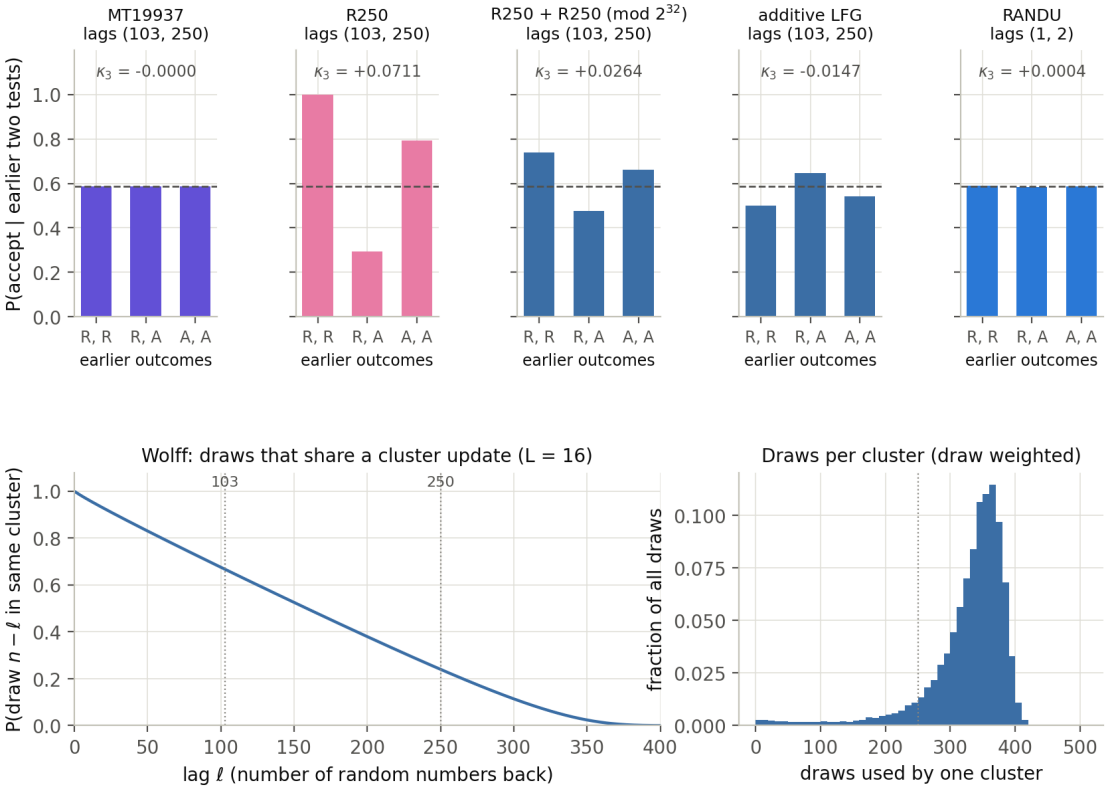


Figure 19: Top: probability that a Wolff bond test is accepted, given the outcomes (R = rejected, A = accepted) of the tests p and q draws earlier. The dashed line is the ideal $p_{\text{add}} = 2 - \sqrt{2}$. R250 accepts with certainty after two rejections. Bottom: fraction of draws whose partner ℓ draws back lies in the same cluster (0.24 at $\ell = 250$), and the distribution of draws per cluster.

A hidden error. The bias is fixed while the statistical error shrinks as $n^{-1/2}$, so the error is invisible in short runs: the running R250 energy stays within 2σ of the exact value up to about 10^5 clusters and becomes unmistakable only after 10^6 (Fig. 20). Without the exact solution, a short R250 simulation would have been accepted.

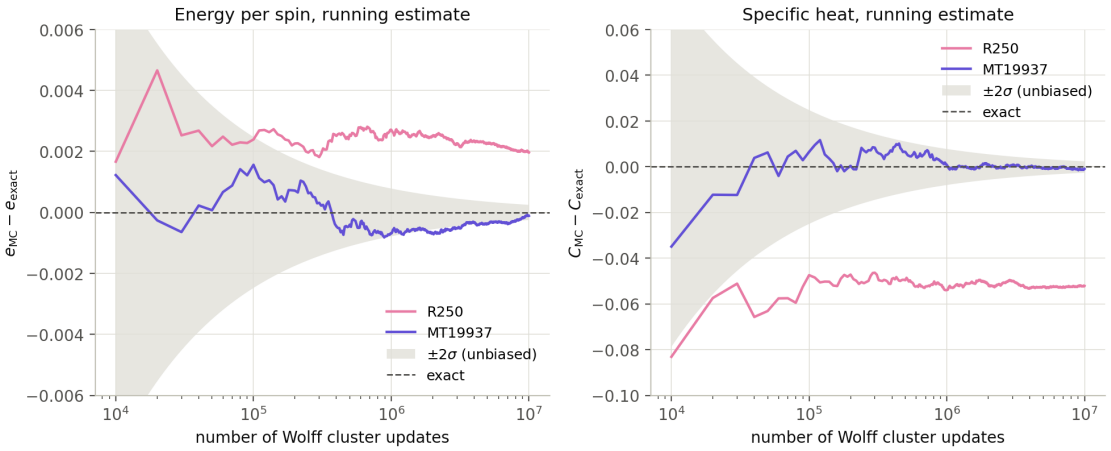


Figure 20: Running estimates of e and C against the number of Wolff clusters for R250 and MT19937. The gray band is the $\pm 2\sigma$ range expected for an unbiased run. The R250 energy stays inside the band until about 10^5 clusters and then settles at a fixed offset.

Remedies, and two traps. Some intuitive fixes do not work (Fig. 21). Using every second or fourth R250 number, or XOR-ing two independently seeded R250 streams, gives exactly the same trinomial recurrence (confirmed with Berlekamp-Massey) and the same bias. Decimating by 3 or 5, or XOR-ing with MT19937, removes it. Adding two R250 streams modulo 2^{32} keeps 38% of the bias, as the reduced κ_3 predicts, and the additive lagged Fibonacci generator with the same lags reverses its sign ($\Delta C = +0.006$, $+4.5\sigma$). Over two-tap registers of increasing length q the bias decays as $e^{-q/265}$ and vanishes for $q \geq 1279$.

Other generators. Wolff updates driven by RANDU, RANDM, MINSTD and `RAND()` are unbiased at 10^7 clusters, although RANDU has a measurable three-point correlation at lags (1, 2), 200 times weaker than R250's. With Metropolis, RANDU and RANDM create a different problem: the generator period 2^{29} is an exact multiple of the 256 draws per sweep, so after one generator period the Markov chain repeats itself exactly, and naive error bars are too small by a factor 2.2. Using only independent periods, RANDM still shows a Metropolis energy bias of -5.8σ with 256 draws per sweep and $+6.1\sigma$ with 257. We have not identified its mechanism and regard it as a tentative result. The negative controls fail badly: the logistic map gives a Metropolis energy 6.4% too low (-107σ).

8 Summary and conclusions

Table 2 collects the decisive anomaly of each generator and the test that exposes it.

Answers to the Part 1 questions.

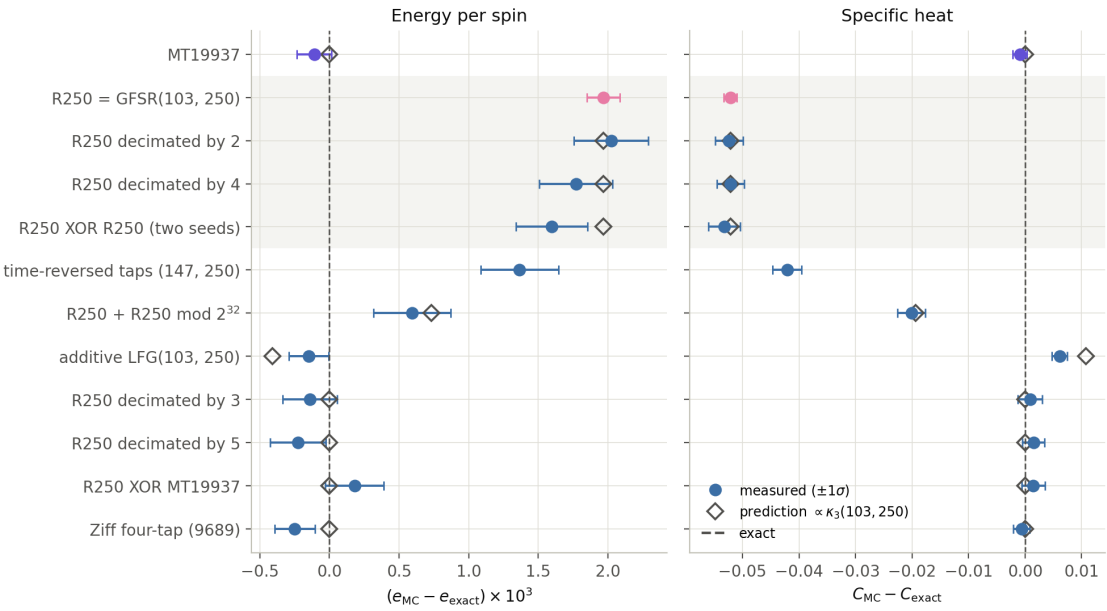


Figure 21: Wolff bias for R250 variants (dots, $\pm 1\sigma$) compared with the prediction that the bias scales with κ_3 at lags (103, 250) (open diamonds). Decimation by 2 or 4 and XOR of two R250 streams obey the same recurrence and keep the full bias; decimation by 3 or 5, XOR with MT19937 and Ziff's four-tap register are unbiased.

Table 2: The most decisive anomaly found for each generator.

Generator	Anomaly	Evidence
RANDU	Triplets on 15 planes; wrong 3D integrals; finite period 2^{29}	$S_3 = 0.0095$; $\hat{\pi} = 3.15426$ ($+60\sigma$); Part 1 test $z \approx -4$
RANDM	Pairs on 6539 lines; lag-one correlation 1/6539; same period as RANDU	$S_2 = 0.19$; $r_1 = 1.53 \times 10^{-4}$ (3.5σ)
RAND	Only 2^{22} output values	58% of 10^7 bins empty; digits fail at $p = 10$
MINSTD	Pairs on 16807 lines; half-period sampling makes bins too even	$S_2 = 0.34$; $D = 0.535$ at $N = 10^9$
R250	Exact three-term XOR identity	Wolff Ising C off by -47σ
RANNUM	None found	passes every test
MT19937	None found (linear complexity 19937 is a known property)	passes every test
MIDSQ	Collapses to 0 or short cycles	at most 16,903 values for any seed
LOGISTIC	Output is a bit shift of the previous output; periodic	cycle of 5.6×10^6; Metropolis -107σ

1. *The difference between RANDM and RANDU.* It is small in the code but large in effect: RANDU is good in 2D and catastrophic in 3D, while RANDM is the worst of all tested LCGs in 2D (with a detectable lag-one correlation $r_1 \approx 1/6539$) and far better than RANDU in 3D to 8D. One-dimensional histograms cannot tell them apart, because both visit exactly the same 2^{29} states.

2. *The seed.* The number of digits is irrelevant. Every trailing zero bit of the seed halves the period of RANDU and RANDM, $\lambda = 2^{29-k}$, so the seed must be odd.

3. *Numerical precision.* Storing numbers in `REAL*4` adds its own, generator-independent artifacts: histograms become too uneven at 10^6 bins and beyond, decimal digits are uniform only to about position 12, and the birthday test sees 0.75×2^{25} effective values. Double precision removes these artifacts but not the generators' own flaws. Precision also matters for the *test*: extracting digits with floating-point arithmetic fails for every generator at position 16.

General lessons.

- Too regular is as anomalous as too irregular. Short-period generators fail by producing histograms, digit counts and spacings that are too even, a failure that the course program's outlier count cannot detect but a one-sided χ^2 can.

- Every anomaly here has a quantitative explanation in terms of the generator's arithmetic: its period, its lattice, its output grid, or a linear recurrence. Each test sees some of these and not others, which is why a battery of different tests is needed.

- Passing distributional tests is not enough. R250 passes the histogram, digit and maximum-spacing tests, yet gives a specific heat wrong by 3.5% in a standard physics simulation, and the error is invisible until 10^6 cluster updates.

- The two modern generators, MT19937 and gfortran's `RANDOM_NUMBER` (xoshiro256**), passed every test we applied. For physics simulations they should be preferred over every other generator studied here.

Limitations. The test families were checked by independent re-implementations of the generators and by validating every test on MT19937 and RANNUM, but the planned second round of adversarial verification was stopped early to stay within the computing budget. A sixth family (a battery of classical tests: runs, gaps, permutations, cell occupancy, and second-level tests) was not completed; only its serial-correlation result is reported. Several smaller effects remain unexplained and should be read as open questions: the RANDM Metropolis bias, a 10% to 25% excess of birthday-spacing collisions for R250 in one and two dimensions, and a weak lag-one correlation in bit 49 of the MINSTD double. Heinrich's original note could not be retrieved, so our maximum-spacing test follows the standard theory of spacings rather than his specific prescription.

References

[1] P. Young, *Randu: a bad random number generator*, Physics 115/242 course notes (2001).

[2] G. Marsaglia, Random numbers fall mainly in the planes, Proc. Natl. Acad. Sci. USA **61**, 25 (1968).

[3] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed. (Addison-Wesley, 1997).

[4] S. K. Park and K. W. Miller, Random number generators: good ones are hard to find, Commun. ACM **31**, 1192 (1988).

[5] S. Kirkpatrick and E. P. Stoll, A very fast shift-register sequence random number generator, J. Comput. Phys. **40**, 517 (1981).

[6] M. Matsumoto and T. Nishimura, Mersenne Twister: a 623-dimensionally equidistributed uniform pseudo-random number generator, ACM Trans. Model. Comput. Simul. **8**, 3 (1998).

[7] D. Blackman and S. Vigna, Scrambled linear pseudorandom number generators, ACM Trans. Math. Softw. **47**(4), 36 (2021).

[8] J. von Neumann, Various techniques used in connection with random digits, Natl. Bur. Stand. Appl. Math. Ser. **12**, 36 (1951).

[9] A. M. Ferrenberg, D. P. Landau and Y. J. Wong, Monte Carlo simulations: hidden errors from "good" random number generators, Phys. Rev. Lett. **69**, 3382 (1992).

[10] U. Wolff, Collective Monte Carlo updating for spin systems, Phys. Rev. Lett. **62**, 361 (1989).

[11] B. Kaufman, Crystal statistics. II. Partition function evaluated by spinor analysis, Phys. Rev. **76**, 1232 (1949).

[12] A. E. Ferdinand and M. E. Fisher, Bounded and inhomogeneous Ising models. I. Specific-heat anomaly of a finite lattice, Phys. Rev. **185**, 832 (1969).

[13] R. M. Ziff, Four-tap shift-register-sequence random-number generators, Comput. Phys. **12**, 385 (1998).

[14] J. Heinrich, CDF/MEMO/STATISTICS/PUBLIC/8032, CDF Collaboration note (2006), as cited in the Particle Data Group review of Monte Carlo techniques (not retrieved).

[15] S. Navas *et al.* (Particle Data Group), Review of Particle Physics, Phys. Rev. D **110**, 030001 (2024), Sec. 42.

[16] R. Pyke, Spacings, J. R. Stat. Soc. B **27**, 395 (1965).

[17] R. P. Brent, An improved Monte Carlo factorization algorithm, BIT **20**, 176 (1980).

[18] J. L. Massey, Shift-register synthesis and BCH decoding, IEEE Trans. Inf. Theory **15**, 122 (1969).

[19] A. K. Lenstra, H. W. Lenstra Jr. and L. Lovász, Factoring polynomials with rational coefficients, Math. Ann. **261**, 515 (1982).

[20] U. Fincke and M. Pohst, Improved methods for calculating vectors of short length in a lattice, Math. Comp. **44**, 463 (1985).

[21] C. Grebogi, E. Ott and J. A. Yorke, Roundoff-induced periodicity and the correlation dimension of chaotic attractors, Phys. Rev. A **38**, 3688 (1988).

[22] P. L'Ecuyer and R. Simard, TestU01: a C library for empirical testing of random number generators, ACM Trans. Math. Softw. **33**(4), 22 (2007).